Erweiterung eines Rahmenprogramms für das automatische Einfügen von Software- Wasserzeichen in Quellcode

Master's Thesis in Computer Science

by **David Schwarzbeck**
born 01.08.1999 in Ansbach

prepared at

Computer Science Department
Chair of Computer Science 2
Programming Systems
Friedrich-Alexander University Erlangen-Nürnberg
(Prof. Dr. M. Philippsen)

Supervisors:
Daniela Novac
Prof. Dr. Michael Philippsen

Started on: 01.12.2023
Submitted on: 01.06.2024

Ich versichere, dass ich die Arbeit ohne fremde Hilfe und ohne Benutzung anderer als der angegebenen Quellen angefertigt habe und dass die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen hat und von dieser als Teil einer Prüfungsleistung angenommen wurde. Alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, sind als solche gekennzeichnet.

Der Universität Erlangen-Nürnberg, vertreten durch die Informatik 2 (Programmiersysteme), wird für Zwecke der Forschung und Lehre ein einfaches, kostenloses, zeitlich und örtlich unbeschränktes Nutzungsrecht an den Arbeitsergebnissen der Master's Thesis einschließlich etwaiger Schutzrechte und Urheberrechte eingeräumt.

Erlangen, den 01.06.2024

David Schwarzbeck

Master's Thesis

Topic: Erweiterung eines Rahmenprogramms für das automatische Einfügen von Software-Wasserzeichen in Quellcode

Background: Wasserzeichen existieren als Authentifizierungsmerkmal für verschiedene Produkte seit dem 13. Jahrhundert. In der digitale Welt hat sich diese Methode auch etabliert, als erstes für Multimediadateien und in den letzten 25 Jahren auch für andere Bereiche, wie zum Beispiel Software. Der Hauptgrund Code zu markieren ist der Schutz vor unerlaubter Wiederverwendung des Programms. Dabei können verschiedene Ziele verfolgt werden: Reverse Engineering, Wiederverwendung von GPL-Code ohne die Bedingungen zu respektieren, bösartige Veränderungen, Software-Piraterie und vieles mehr. Wasserzeichen können dazu dienen, solche Sicherheitsziele zu erreichen. Es gibt zwei Arten von Wasserzeichen für Code: statische und dynamische. Statische Wasserzeichen beeinflussen die Funktionalität des Programms nicht und werden wie eine Signatur innerhalb des Codes auf der syntaktischen Ebene hinzugefügt. Die Wiedererkennung benötigt die Ausführung des Programms nicht. Dynamische Wasserzeichen funktionieren wie sogenannte Ostereier (Easter Eggs), sie sind also eine versteckte Funktionalität, die nur dann erkannt wird, wenn das Programm ausgeführt wird, meistens mit einer geheimen Eingabe. Ein Wasserzeichensystem wird meistens nach den folgenden Kriterien evaluiert: Glaubwürdigkeit (wie stark es den Urheber identifiziert), Robustheit (wie schwer es ist, das Wasserzeichen zu entfernen), Transparenz (wie schwer es zu erkennen ist, dass der Code ein Wasserzeichen enthält), Widerstandfähigkeit (wie kompliziert es ist, die Wiedererkennung des Wasserzeichens zu kompromittieren), Datenrate (Informationsgehalt des Wasserzeichens), Laufzeit (ob und um wie viel das Wasserzeichen das ursprüngliche Programm verlangsamt) und zusätzlicher Speicherbedarf.

Task: Ziel dieser Arbeit ist es, ein bestehendes, auf LLVM basierendes Rahmenprogramm zu erweitern, das beim Übersetzen automatisch Wasserzeichen in den Code einfügen kann. In der Literatur sind eine Reihe von Wasserzeichensysteme bekannt. Zwei davon sind bereits Bestandteil unseres Systems. Im Rahmen der Arbeit sollen zwei weitere (z.B. als Plugins) realisiert werden, in der Regel als AST- der Zwischencodetransformationen. Die umgesetzte Methoden werden dann anhand von vorgegebenen und noch zu ergänzenden Programmsammlungen quantitativ bezüglich der oben genannten Kriterien evaluiert und mit dem bereits vorhandenen Verfahren verglichen. Bei Interesse sprechen Sie uns an. Wir identifizieren dann gemeinsam mit Ihnen die umzusetzenden Wasserzeichensysteme, Benchmarks, etc. Diese Arbeit kann als Bachelor- oder als Masterarbeit ausgestaltet werden.

Supervisors: Daniela Novac, Prof. Dr. Michael Philippsen

Author: David Schwarzbeck

Abstract

Software Watermarking allows to provide ownership information in a program without limiting the users freedom to use the software in the general case. Since all Software Watermarking techniques have their own strengths and weaknesses, it is important to employ a combination of several schemes. This thesis presents three novel dynamic software watermarks that can be automatically inserted into programs using the infrastructure of the LLVM compiler. This allows automatic insertion of the three methods into programs by simply specifying a watermark message during compilation, making their combination and inclusion as simple as possible. The message is encoded as a key-value pair and the inclusion procedures add a transformation from the key to the value to the program, s.t. if the key is present in the dynamic environment of the program, the value will be generated. The *Register Expansion Watermark* facilitates casts of 32-bit values to 64-bit values to use the additional 32-bits to encode the watermark. The key is inserted via a debugger and the value can then be detected in the upper bytes of the registers. The watermark shows a small overhead in the final program and can be inserted without many preconditions, however it suffers from a small throughput size and can be removed automatically. The *SemaCall* Watermark uses functions for which the semantic of the parameters and return value are known, like the functions of the standard library of the language. The watermark then detects the key in the dynamic environment and triggers the generation of the value, that is then displayed in an easter-egg procedure. It shows good performance and arbitrary throughput, however false-positive triggering of the easter-egg is possible. The *Sidedata* Watermark uses the same technique for stack-allocated arrays with respect to the allocation size.

Kurzzusammenfassung

Wasserzeichen in Software erlauben es Urheber Informationen in einem Programm einzubetten, ohne die Freiheit des Nutzers einzuschränken, die Software allgemein zu verwenden. Wasserzeichen Techniken haben jeweils eigene Stärken und Schwächen, entsprechend ist es notwendig eine Kombination mehrerer einzusetzen. Diese Arbeit präsentiert drei neue dynamische Software Wasserzeichen Techniken, die automatisch in Programme über die LLVM Übersetzer Infrastruktur eingefügt werden können. Dies erlaubt das automatische Einfügen der Wasserzeichen, indem während des Übersetzens die Wasserzeichen-Nachricht spezifiziert wird. Das macht das Einfügen und Kombinieren der Wasserzeichen Techniken so einfach wie möglich. Die Nachricht ist als Schlüssel-Wert Paar kodiert und die Einfügeprozeduren fügen Transformationen vom Schlüssel zum Wert ein, sodass der Wert im fertigen Programm generiert wird, wenn der Schlüssel in der dynamischen Umgebung präsent ist. Das *Register Expansion* Wasserzeichen benutzt Erweiterungen von 32-bit Werten zu 64-bit Werten um in den oberen 32-bit das Wasserzeichen zu generieren. Der Schlüssel wird mittels Debugger eingefügt und der Wert wird daraufhin in den oberen bits der Register generiert. Die Technik hat einen geringen Overhead auf die schlussendliche Laufzeit des Programms, das Wasserzeichen kann aber automatisch entfernt werden. Das *SemaCall* Wasserzeichen benutzt Funktionen deren Parameter und Rückgabewert Semantik bekannt ist, wie die Funktionen der Standardbibliothek. Der Schlüssel kann dann aus der dynamischen Umgebung erkannt werden und ein „easter-egg“ auslösen, das den Wert generiert und als Seiteneffekt ausgibt. Die Technik ist performant und erlaubt beliebig große Nachrichten, die Erkennung des Schlüssels kann aber das „easter-egg“ fehlerhaft auslösen. Das *Sidedata* Wasserzeichen benutzt die selbe Technik auf Arrays die auf dem Stack allokiert wurden.

Contents

1 Introduction **1**

2 Related Work **3**

3 Methodology **5**
3.1 Notation 5
3.2 Testing 7
3.2.1 Test Suite 7
3.2.2 Test Environment 8
3.3 Implementation of Watermarks 8
3.4 Evaluation of Watermarks 9

4 Watermarks **11**
4.1 Register Expansion Watermark 12
4.1.1 Implementation 14
4.1.2 Detection & Removal 23
4.1.3 Evaluation 26
4.2 SemaCall Watermark 32
4.2.1 Implementation 33
4.2.2 Detection & Removal 48
4.2.3 Evaluation 50
4.3 Sidedata Watermark 59
4.3.1 Implementation 59
4.3.2 Evaluation 63

5 Conclusion & Future Work **67**

1 Introduction

Piracy and copyright infringement worldwide cost billions of dollars each year. According to the BSA Global Software Survey [1], 39% of all software installations in 2015 worldwide have been unlicensed, making up a commercial value of 52 billion dollars in one year. There exist multiple mechanisms to protect software from piracy or to prove that a piece of software has been used in another project without permission. One of these techniques is software watermarking.

Software watermarking describes a set of techniques where a message - the *watermark* - is inserted into a cover message. The watermark should be hard to remove and should identify the author of the original information s.t. it functions as a copyright notice and protection. The watermark allows to prove legal ownership of the message and - if visible - discourages unauthorized distribution or other kinds of illegal use of the message.

There exist many broadly adapted techniques for media watermarking like images [25] or videos [2] and techniques for embedding watermarks into software have been developed as well [3] [18]. There are other software protection techniques that are already widely adopted, e.g. DRM [16], which uses digital locks that prevent unauthorized use of digital content. According to CBC Newsin "The pros, cons, and future of DRM", it often fails in scenarios where digital content is locally copied for private use, used for research purposes or public use like in libraries. Watermarks circumvent such caveats by providing a proof method for legal actions against copyright infringement, without limiting the use of digital content. Additionally, watermarking can be used to verify the program's integrity.

According to Collberg and Thomborson in their paper "Software Watermarking: Models and Dynamic Embeddings" [3], software watermarks can be divided into two categories:

- Static Software Watermarks include the watermark inside the executable e.g. in the data segment of the assembly. The watermark does not change during program execution. While there are tamper-proofing methods, the watermark can be easily attacked, distorted, or removed.
- Dynamic Software Watermarks encode or generate the watermark during the execution of the program in its dynamic state. It may be generated, altered and destroyed during program execution. This makes attacks and removal of the watermark generally harder.

Because of the advantages of dynamic software watermarks, we will focus on this type

of watermarks. Collberg et al. [3] propose categories to distinguish different kinds of dynamic watermarks:

- Easter-Egg Watermarks have a procedure - the "easter-egg" - that is triggered by a special, unusual prompt or dynamic state. The watermark is encoded or produced in the triggered procedure and usually salient to the user.
- Dynamic Data Structure Watermarks encode the watermark, usually for a specific input, inside the dynamic state and memory of the program during runtime. They usually don't produce any output and are detectable with a debugger or specific extraction techniques.

While each watermark can be inserted, detected, or removed manually by the programmer, a specific insertion, detection, and removal algorithm could be written to automate this process. It is sensible to extend an existing compiler since this allows the reusage of many procedures for program parsing, code optimization and generation. The LLVM Project [20] provides a large compiler environment for many languages and is extensible by plugins.

Even though the concrete insertion and implementation of each watermark is fairly different, all of the watermarks presented in this thesis share a common infrastructure for inclusion in LLVM. Since working on the Intermediate Representation (LLVM-IR) of the programs suffices to insert watermarks in most cases, all watermarks are implemented as passes in the optimization pipeline of the new pass manager. They transform the Intermediate Representation and afterward allow further optimization of the code.

Developing new dynamic software watermarking techniques is important, since no watermarking technique is watertight and each has its own strengths and weaknesses. Choosing a watermark that fits the scenario of a specific program or combining several different watermarking techniques, which may be able to balance out the weaknesses of each other, can improve the effectiveness of the included watermark. For software watermarking in practice, a solution that employs several watermarking techniques to benefit from the synergy of the combination will almost always be desirable.

We will explore various ideas for watermarks inside the mentioned categories, the implementation of their inclusion procedure inside of the LLVM-Middleend, and evaluate their advantages and disadvantages in practice. The following chapter presents reference research conducted in the field and a classification framework to place our work in its context. The methodology in chapter 3 describes the infrastructure and techniques used to develop and test the watermark insertion procedures with LLVM. Chapter 4 presents three novel dynamic watermarks, their implementation and evaluates them. A final conclusion summarizes the evaluation results of each watermark in context and provides an overview over the future work that can be conducted to further examine the watermark procedures or to improve them.

2 Related Work

A work that summarizes the state of software watermarking at that time is "Software Watermarking: Models and Dynamic Embeddings" by Christian Collberg and Clark Thomborson [3] from 2000. They composed criteria to evaluate the quality of software watermarks, attack types against which they have to be hardened (additive attacks, subtractive attacks, and distortive attacks) and categories into which the watermarks can be divided (e.g. dynamic vs static watermarks, Easter Egg Watermarks, Data Structure Watermarks, etc.). Those criteria and categories will be revisited in Methodology (section 3). Additionally, they explained the Dynamic Graph Watermarking technique, which embeds the watermark in graph structures of the program, possible attacks on this kind of watermark, and possible tamper-proofing methods to prevent some attacks.

Jens Palsberg, Sowmya Krishnaswamy, Minseok Kwon, Di Ma, Qiuyun Shao, Yi Zhang present a new form of watermark for Java programs in their paper "Experience with Software Watermarking" [18]. After revisiting the necessity and usefulness of dynamic software watermarking, they explain how they insert their watermark in the form of integers into Java classes. The watermark is represented by a Planted Plane Cubic Tree (PPCT), a special form of binary trees. Such trees allow the embedding of integer values. This data structure is added to classes in the Java representation and encodes the watermark integer. This allows retrieval of the data structure during runtime, especially if the watermark is known, and therefore constitutes an interesting implementation of a dynamic data structure watermark. Their experiments showed a small heap and performance overhead. They tried many tamper-proofing techniques with this watermark like randomization, interweaving of the watermark code with the original code and obfuscation.

Daniela Novac, Christian Eichler and Michael Philippsen propose in their paper "LLWM & IR-Mark: Integrating Software Watermarks into an LLVM-based Framework" [17] a framework for multiple watermarks that are automatically inserted into software with the LLVM compiler, which enables and helps the more widespread use of software watermarking by making their insertion easier and packageable into one framework. This is enabled by working on the LLVM Intermediate Representation (LLVM IR), which allows expansion by providing plugins that constitute e.g. transformation passes to include the watermark insertion into the bytecode of the program. Additionally, they summarize multiple watermarks that are included in the framework. Those range from embedding integers into the control flow graph (WaterRPG) over encoding the watermark by re-

placing jumps with a branch function (Self-validating Branch-based Watermarking) to using the Chinese remainder theorem to encode the watermark by hash functions that calculate constants during runtime. Additionally, the adaptions to these watermarks to make them work on the LLVM Intermediate Representation are described. We will further expand this framework by working on the LLVM IR and exporting the inclusion transformations for our watermark as plugin libraries.

Many more papers present dynamic watermarking techniques. However, since we are working on novel techniques the work on categorizing software watermarks, providing a general notation and scientific framework for testing and describing these watermarks additionally to tamper-proofing methods as well as description of possible attacks are more relevant for this work.

3 Methodology

3.1 Notation

Terms and concepts:

- **Basic Block**: a sequence of instructions, without branches except for the last one
- **Control Flow Graph (CFG)**: a graph where the nodes make up basic blocks and the edges connect the basic blocks with their entry and exit branches, i.e. each branch in each basic block is per definition the last instruction and therefore connects two basic blocks or nodes in the graph.
- **Data Flow Graph (DFG)**: a graph where the nodes represent variables or the result of operations. The edges represent the data flow, especially from variables to operations.
- **Intermediate Representation (IR)**: a compiler often compiles the high-level language to an intermediate representation, that constitutes its own kind of language. It is usually low-level and similar to assembly code. Optimizations and transformations are run on this representation. Afterwards, the Intermediate Representation is compiled to the target assembly.
- **Static Single Assigment Form (SSA)** A form of the intermediate representation, where every variable has exactly one definition. To convert ordinary code with e.g. loop-counter variables, Phi-functions are used. Phi-functions return for each incoming Basic-Block a corresponding variable, i.e. the function chooses one of the variables in its arguments depending on the control-flow.

Algorithms are denoted in a pseudocode manner, to be easily understandable. However, they of course follow a common notation, control-flow structures, and data types:

- For most procedures, common C-like control structures like for- and while-loops, if-else conditions, and function calls are used.
- Types are denoted in a functional manner with a colon : after the variable name, i.e. $x : Int$. The used types of course match the topic and are more compiler construction-specific, like *Function* for function definitions in LLVM-IR, or *Operand* for operands in instructions in LLVM-IR.

- Variable assignments are denoted with a $\leftarrow$, e.g. $x \leftarrow 7$ assigns 7 to x.
- $\bot$ and $\top$ are the usual boolean types ($\bot$ for false, $\top$ for true), however $\bot$ is sometimes used in the context of partially ordered sets and represents a "least-element", most often used in place of `NULL` in C: a value to indicate that something has not been initialized or found yet.
- Common mathematical notations are used as well, e.g. $x \in l$ to indicate that x is part of l (e.g. if l is a list or set, it contains x), union $\cup$ or intersection $\cap$ for containers, as well as the "without" notation (e.g. $l \backslash x$ "l without x").

To avoid literal evidence of the watermark values in the binary, the watermark is encoded as a key and value pair. The key is a prompt, that has to be present in some form in the dynamic state of the program. The inserted procedures then generate the value from the key. I.e. neither the key nor the value are included in the binary, only mechanisms to optionally detect the key and to transform the key to the value.

We will use the notion of cryptographic hash functions or transformations. They are most often needed to provide a transformation procedure from a key to value to e.g. generate the watermark from a special input. The general case is that a variable is inserted in a hashing procedure that generates an output value, which is then tested on a value that would have been produced if the watermark key was inserted. This is necessary because we can't directly test for the key without compromising it by literally including it in the bytecode. This should fulfill three criteria:

- Non-Reconstructable: The key should not be easy to reconstruct from the value. E.g. if an integer generated from dynamic information by such a hash procedure is tested on a specific value, it should be hard or impossible to uniquely reconstruct the original input data.
- Uniqueness: Each hashing procedure should be different s.t. the key candidate set (because the key should not be uniquely reconstructable, we assume it is possible to generate a set of candidates that would produce a value that matches the test) differs from insertion point to insertion point. This makes it harder to attack all hashing functions at once, since each may generate different values.
- Invisibility: The hashing procedure itself should contain no evidence of the watermark values and should look like "normal" code.

This can be achieved by constructing functions where the domain is larger than the codomain i.e. the function should not be injective, s.t. for $f(x) = y$ for a given y, x is not unique. This can be achieved by using modulo instructions (e.g. for $x \ mod \ 10 = 5$, x is not unique, and the candidate set is infinite $x \in \{i \cdot 10 + 5 \mid \forall i \in \mathbb{N}_0\}$). Additionally, random arithmetic instructions are generated and applied to either the variables that could hold the key (the random instructions then constitute a cryptographic hashing

transformation by theirself if at least one modulo operation is included) or to the output of a fixed string hash function, to guarantee the Uniqueness criteria.

3.2 Testing

3.2.1 Test Suite

We compiled a set of compatible C and C++ test programs, each with an expected output and exit code. The suite contains test programs from multiple sources:

- We took the *single-exec suite* from the *c-test-suite* repository [26], containing many single *.c* files that should terminate with the exit code 0 and some with expected output. The repository was released under the MIT License. The source code of the programs usually only consists of a few lines of code and is used to test some language-specific features. They rarely use libraries or interactions with the system, at most print statements or simple arithmetic operations.

- We selected compatible tests and benchmarks from the *llvm-test-suite* repository [21], containing a large variety of test cases of different languages. We selected the C and C++ single-source test cases (respectively in the folder *SingleSource*) that work with clang version 16.0.6 (and therefore adhering to the recent C and C++ standards without supporting extensions). The included test cases range from reduced versions of programs that triggered a bug in the clang or LLVM compiler to benchmarks that test the performance of the vector implementation in C++. Test cases that evaluate the semantics of multithreading, floating point exceptions, the semantics of cast operations, and so on are included.

In order to create a simple Makefile, only single source programs were selected. Tests that were created to test the C or C++ frontend (i.e. special type behavior, template implementation, etc.) were eliminated since the main goal is testing the LLVM-IR and the insertion of watermarks in it. For simplicity, we also excluded tests needing special arguments or input from the terminal.

The final selection contains 426 C and 70 C++ test files from which 93 are benchmarks, which are also test cases, but especially time-consuming ones for profiling the compiled programs.

The test suite is used for verifying the semantic correctness of the watermark, i.e. checks that inserting the watermarks does not alter the program's semantic behavior. Of course, the tests only give a small indication of the correctness of the watermark and do not prove it, but additionally, to the argumentation of correctness of the watermark insertion algorithms, it indicates a presumably correct implementation of it. The concept of each watermark insertion algorithm in itself should nevertheless be correct by construction.

The benchmarks additionally give an idea of the performance toll of each watermark in the compiled program, while the compilation of the full test suite indicates the performance toll of the insertion of each watermark during compilation. The tests are run by a simple runner script that can differentiate between the benchmarks and the other test cases and checks for correct behavior.

Additionally, the GNU Core utilities [6] and the LLVM project [20] itself are used to test in how many places of a "real-world-project" the watermarks can be included, instead of just testing on a test-suite with minimal example programs and benchmarks.

3.2.2 Test Environment

The tests were all compiled by the clang compiler (*clang* and *clang++*) in version 16.0.6. The used (and for the watermarks patched) version of *opt* (The LLVM Optimizer) is custom compiled from the release/17.x tag in the LLVM repository [20].
All compilations and test cases are run in an up-to-date version of Arch Linux on a system with a Ryzen 9 7900X with 32GB Ram but are executed in a single thread.

3.3 Implementation of Watermarks

The passes are directly included in the registry of passes in LLVM and are therefore baked into a patched version of *opt*. Because this is by far not the most portable approach, for each watermark one dynamic library that can be loaded into an already existing version of *opt* was created, which then provides the same functionality as the patched version of *opt*. It is therefore not necessary to provide or compile a patched toolchain version, but the single dynamic library suffices.
The implementation in the optimization pipeline of *opt* is sensible since it allows inclusion of the watermark into an intermediate representation disjoint from the targeted frontend language or backend assembly. One disadvantage is, that it complicates a normal compilation pipeline. The inclusion of the watermark splits the normal compilation into two steps and happens in between:

1. Compile frontend language to LLVM-IR (e.g. `clang -emit-llvm -S file.cpp -o file.ll -O1`)

2. Optimize intermediate representation and insert watermark (e.g. `opt -load WatermarkPlugin.so -passes="watermark-pass" -S file.ll -o file.ll`)

3. Compile intermediate representation to target binary (e.g. `clang -o file file.ll -O3`)

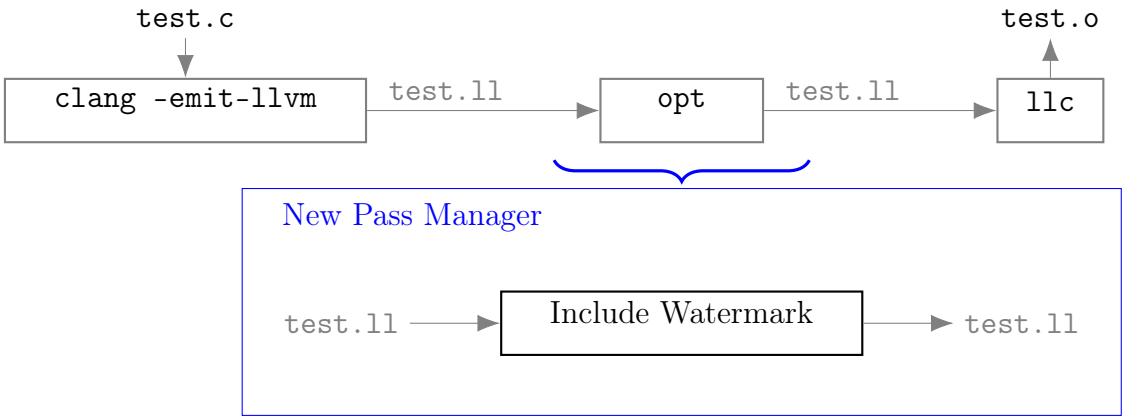


Figure 3.1: Compilation Pipeline with the Watermark as a Transformation Pass

It is also possible to patch a custom version of clang, that manually includes the watermark transformation passes. However, this requires more manipulation of the source code but can be the preferred option in practice, since this allows direct compilation with CMake or other build systems that already support the clang compiler. A patched version of clang is included with the example implementations of the watermarks.

3.4 Evaluation of Watermarks

It is important to measure the effectiveness of the watermark in various aspects. It must be included in the program well enough to be detectable in a credible way, but shouldn't be easy to detect or remove from an attacker. It must not affect the program's normal behavior, should have a low memory and performance toll, and should have enough throughput to insert identifying data.
We therefore evaluate the watermarks along the following six criteria, proposed by Collberg et al. in "Surreptitious Software: Obfuscation, Watermarking, and Tamperproofing for Software Protection" [15]:

- Credibility: How much the watermark identifies the author.
- Robustness: Difficulty to remove the watermark.
- Transparency: How easy it is to detect the watermark.
- Resilience: Difficulty to compromise the transparency with attacks.
- Throughput: How much information the watermark can store.
- Runtime & Memory consumption: Runtime and Memory toll of the watermark during program execution.

It is also important to evaluate the effectiveness of various kinds of attacks on each watermark, as proposed by Collberg and Thomborson in "Software Watermarking: Models and Dynamic Embeddings" [3] on the following types:

- subtractive attacks: removal of the inserted watermark. Effective if a complete removal without program corruption is possible.
- distortive attacks: distortive transformations that may distort the detectability of the watermark. Effective if the watermark is no longer recognizable, but the program still carries value to the attacker.
- additive attacks: add different watermarks that overwrite the present one. Effective if a complete overwrite is possible.

4 Watermarks

4.1 Register Expansion Watermark

On most 64-bit CPUs the registers each have a 32-bit, and a 64-bit variant where the 64-bit variant overlaps the others as stated in the AMD64-Assembler-Manual [5]. This makes it possible to exchange each 32-bit register by a 64-bit one, as long as the value is narrowed down to 32-bit again before its usage. The expansion of a register should not come with too much performance overhead since even for 32-bit operations the same machine register is used and most modern-day CPUs are optimized for 64-bit operation. Doing so, the upper 32 bits are unused during the registers lifetime. This is similar to unused bytes in an aligned structure caused by padding. Because CPUs expect addresses to be a multiple of their underlying architecture data size, additional bytes are inserted as padding to ensure that each address of each attribute starts e.g. at a multiple of 4 bytes. It would be possible to insert a watermark inside this padding, however the size of the padding may vary for each structure. Therefore we will concentrate on the unused upper bytes of the expanded registers. This allows to include a 32-bit watermark inside this padding:

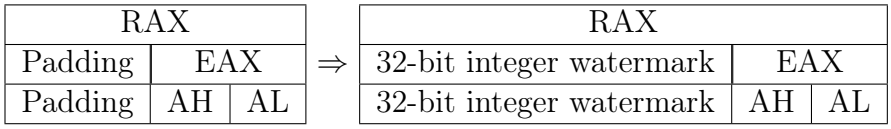


Figure 4.1: Illustration of 64-bit register padding and position of watermark

But doing the inclusion statically brings the same disadvantages as all static code watermarks experience, according to Collberg et al.[3]: A static watermark manifests itself not only during runtime but in the binary as well, which allows easy detection and removal. It is affected by many obfuscation methods. Calculating the watermark instead only during the runtime of the affected program and interweaving the calculation with already present instructions, would make removal more complicated.

The idea is to define the watermark as a key-value pair, each as a 32-bit integer (the value because the padding is 32-bit wide and the key because calculating a 32-bit value is arguably the easiest from another 32-bit key). The value should manifest itself only if the key is present, it is therefore necessary to modify the program in a way that if the key has been placed in a predefined place, the program calculates the watermark value from it and places it in the padding of the registers.

A sensible simplification is to narrow the context down to functions. Here the inputs capable of receiving the watermark key are the 32-bit integer function parameters and since it is possible that a function is called with a 32-bit user input or to run the program in `gdb` and manipulate the parameters manually, it is possible to trigger the watermark.

Of course, it would also be possible to assume that all 64-bit parameters contain the watermark already in their upper bits and to expand all 32-bit parameters to 64-bit, this would allow a broader inclusion and enable inheritance of the watermark between functions (function a calls function b with a 64-bit parameter that already contains the watermark). However, all calls to patched functions with 32-bit parameters would have to be changed to cast the parameters to 64-bit beforehand. User input could then only provide the key if it is parsed to a 64-bit value, 32-bit variables could then only carry the watermark if it was inserted with a debugger. Therefore we only assume the watermark key to be present in the 32-bit integer parameters.

Additional instructions have to be added, which generate the watermark value from the watermark key in the parameters. Since generating the watermark does not include it in existing calculations or used values, it is necessary to modify other variables and instructions s.t. the generated watermark value is propagated from variable to variable with their calculations. This not only interweaves the watermark into existing calculations but also increases the places where it can occur. To accomplish this, already present connections in the data-flow graph are modified to preserve the watermark along calculations, i.e. the watermark can be inherited if a variable with the watermark in its upper bits is used in another calculation. Including the watermark in the result of an existing 32-bit integer instruction can therefore be categorized into two cases:

- If the instruction accesses another value that already contains the watermark in its upper bits, the effect of the instruction with the other operands on the upper bits has to be counteracted. E.g. an addition of a variable x, which contains the watermark, and another variable y only needs y's upper 32-bits to be 0 for the watermark to propagate from x to the result. If y has been a 32-bit value, that has been expanded to 64-bit by an unsigned casting instruction (s.t. negative values also have only 0s in the upper bits), this is already the case. If y is a 64-bit value (which may be the case, if it carries the watermark as well) the upper 32-bits have to be set to 0 by an additional instruction like a binary and with a constant.

- If the instruction accesses a 32-bit integer parameter, the parameter value (assumed to contain the watermark key) has to be transformed to the watermark value, shifted by 32-bit and added to the result of the instruction. The result then carries the watermark, however this comes at the cost of a key-to-value transformation.

For each case, the result of the instruction has to be expanded to 64-bit beforehand. Therefore the watermark can be inserted into any 32-bit integer variable that accesses a parameter or another variable that already includes the watermark by expanding it and adding new instructions. The watermark fits the definition of a Dynamic Data Structure Watermark by Christian Collberg and Clark Thomborson [3] since it employs existing data structures (the variables are mapped to registers and the stack) and inserts the watermark depending on dynamic inputs. This makes it affected by obfuscation attacks like splitting of variables [3] as well; To be specific: not the watermark is affected but

the detection of it in a debugger, since the variable is in some way split into multiple pieces that have to be put together beforehand. Doing so hurts the credibility of the watermark.

The insertion procedure fits into the optimization pipeline as a transformation since the complete function can be accessed and each variable has exactly one definition (because of the SSA form, see [19]), making variable analysis and transformations easier. Since the watermark insertion can extend the types of variables to expand them, it is not needed to modify the final assembly code - the intermediate representation is low-level enough to analyze the variable padding, which will then be mapped to the described register by a LLVM backend. It is then necessary to analyze if optimizations beforehand hinder the insertion of the watermark (which is probable since function-inlining at least decreases the number of functions in which the watermark can be inserted) and if optimizations afterward destroy or remove the watermark.

4.1.1 Implementation

The implementation algorithm consists of three stages: marking of the variable candidates, expanding them to 64-bit, and insertion of the watermark. To visualize the transformations, the data flow graph of the following example is added:

```
int f(int a) {
 int b = a + 7;
 int c = b * 2;
 int d = b + c;
 return d;
}
```

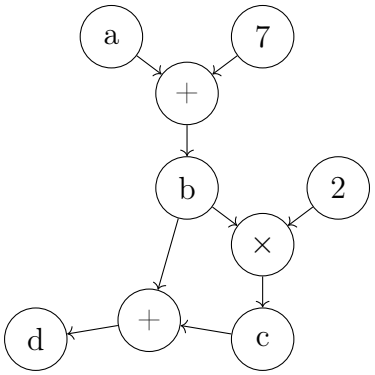


Figure 4.2: Example code and data flow graph for the transformation of the Register Padding Watermark

1. **Marking of Variable Candidates**:

   A fixpoint algorithm is used to mark those variables that are capable of holding the watermark in a function F. A set S of those candidates is initialized with all 32-bit integer parameters of that function (remember that the actual candidates must not contain function parameters since they have to remain unchanged). For each 32-bit integer instruction, it is checked if one of its operands is already contained in the set. If this is the case the variable is added to the set as well. The iteration over the remaining, non-included variables is repeated as long as S changes during

the iteration. The insertion of the watermark is limited to a set of instructions containing addition, subtraction, multiplication, logical and, logical or and Phi functions and selects. Therefore only values from instructions of those types are included in S. The final candidates are the ones in S that are not parameters.

Figure 4.3 shows color-coded from blue (initial state) to red (last iteration) in which iteration each variable was added to S in the data-flow graph from figure 4.2. a is added in the initial state, because it is a parameter. In the 2nd iteration b was added, which generates the watermark value from the key in a. In the 3rd c and in the last one d is added, each inherit the watermark value from b. All candidates are capable of receiving the watermark in some way through their operands: the ones only connected to the parameters have to calculate the watermark value from the key and add it, and the others have to preserve it from one of their parameters.

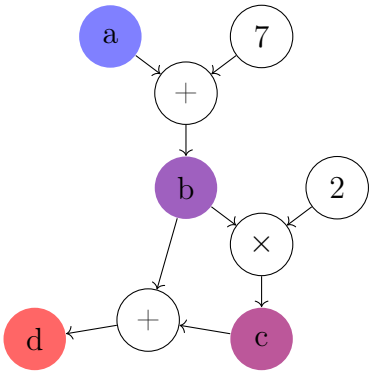


Figure 4.3: Color grading of the data flow graph from blue to red of the iteration in which each node was added as a candidate for the Register Expansion Watermark

The corresponding fixpoint algorithm in pseudo-code is given in Algorithm 1.

Algorithm 1 Marking of Candidates for Register Expansion

```
procedure MARKCANDIDATES(F:Function)
    S ← parameters(F)
    changed ← ⊤                                  ▷ Indicates that S was changed
    while changed do
        changed ← ⊥
        for all i : Instruction ∈ instructions(F) do
            if type(i) ∈ {ADD, SUB, MUL, AND, OR, PHI, SELECT} then
                for all o : Operand ∈ operands(i) do
                    if o ∈ S then
                        S ← S ∪ {i}
                        changed ← ⊤
                    end if
                end for
            end if
        end for
    end while
return S\parameters(F)
end procedure
```

2. **Expanding Candidates to 64-bit**:
   All marked candidates are expanded with a cast from 32-bit to 64-bit after their calculation (as an additional value). Another cast from 64-bit to 32-bit is inserted and each usage of the candidate is replaced by this cast. The first cast will later be replaced by the calculation of the watermark from its operands and the second cast is needed for all instructions that reference this candidate and are not going to be expanded. The reason for introducing the first cast and using the 32-bit cast even for candidates that could be directly used in their 64-bit version is, that possibly not all candidates in its operands and other candidates that reference this candidate have already been expanded and still need a 32-bit version. The first cast works therefore as a dummy for the later calculation.

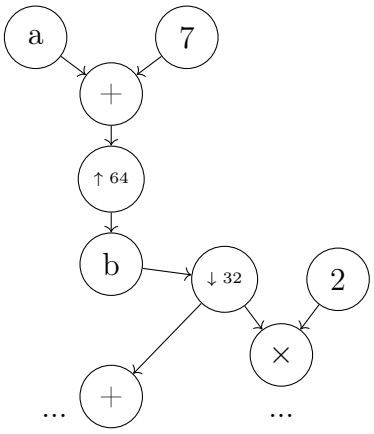


Figure 4.4: Visualization of the data flow graph for the expansion of the candidates to 64bit for the Register Expansion Watermark

In figure 4.4 the variable b was expanded, the cast to 64-bit inserted as the ↑ 64 node and the usage cast to 32-bit as the ↓ 32 node.

Algorithm 2 Expanding Candidates to 64-bit for Register Aliasing

```
procedure expandCandidates(F : Function)
    S ← markCandidates(F)
    for all exp : Instruction ∈ S do          ▷ exp is a 32-bit integer candidate
        v ← cast(exp, 64)
        c ← cast(v, 32)
        replaceUsage(exp, c, F)
    end for
end procedure
procedure replaceUsage(a : Instruction, b : Instruction, F : Function)
    for all i : Instruction ∈ instructions(F) do
        for all o : Operand ∈ operands(i) do
            if o = a then
                replace a by b in i
            end if
        end for
    end for
end procedure
```

While this algorithm may be altered to work without the interim cast, this would require dealing with special cases of not-yet-expanded operands and already-expanded references to this candidate. For simplicity, the cast from 32-bit to 64-bit is included in this step and removed afterward in the next.

3. **Insertion of the Watermark**: In an iteration over all expanded candidates it is checked if the candidate has another candidate as one of its operands. If it has not it must have a parameter in its operands, because of the marking-of-candidates step.

   - If the candidate only contains parameters and no other candidates in its operands, a transformation from the key (assumed to be contained in all 32-bit integer parameters) to the value has to be generated. The most simple approach is straightforward: the difference between the watermark value and the key is added to the parameter.

     The downside of this is, that instead of including the watermark key or value in the binary, this includes their literal difference, which helps detect the watermark already in the bytecode. To make it more difficult to detect the watermark the generation can be modified with some random factors (randomization as an obfuscation technique [18][15]): Multiple instructions like division, subtraction, and addition with random values and the parameter can be randomly chosen to be added to the transformation. In any case, at least one modulo operation has to be inserted to fulfill the criteria for hashing transformations proposed in Methodology (see chapter 3). In the end, the difference that remains still has to be added but should be each time another number, making it more difficult to identify it as the watermark transformation. The final difference can be calculated during insertion since the parameter is assumed to contain the watermark key, yet the remaining optimization passes can't delete those instructions with constant folding.

Algorithm 3 Transformation of watermark key to value

```
procedure TRANSFORMKEYTOVALUE(P : Parameter)
    lastInstr ← P
    value ← watermark value
    for n times do                          ▷ n is the number of random instructions
        lastInstr, value ← createRandomInstruction(lastInstr, value)
    end for
    lastInstr ← add(lastInstr, watermark key - watermark value)
end procedure
```

After the actual watermark has been calculated it can be shifted by 32-bit and added to the cast to 64-bit that was created in the expanding step.

- If the candidate contains another candidate in its operands we can replace the cast to 64-bit by an actual 64-bit calculation with the 64-bit versions of the operands instead of the down-casted versions (for those, that are not candidates, we must insert a prior conversion from 32-bit to 64-bit) and make sure the watermark in the upper bytes persists. This can be done by manipulating the 64-bit-versions of the operands in advance with additional instructions. E.g. an addition between two variables $z = x + y$ with the 64-bit versions of the variables as x' and y', where the watermark is contained at least in x', z can be expanded to z' with $z' = x' + (y' \ \& \ \texttt{0x00000000FFFFFFFF})$ (i.e. by setting the upper bytes of the second variable to 0).

Before showing the algorithm it is relevant to explain how the insertion deals with the second case for the selected instruction types:

- **Addition**: As already mentioned, one operand containing the watermark (i.e. one of the expanded candidates) is chosen. All other operands are combined with a constant in an and-operation to set the upper bytes to 0, s.t. they don't interfere with the watermark in the upper bytes of the selected operand.
- **Subtraction**: If the first operand contains the watermark, the same procedure as with addition can be applied. If the second operand contains the watermark we copy the upper 32-bits of the second operand to the first and then set the upper operands of the second operand to 0, resulting in a subtraction of 0 from the watermark in the upper bytes.
- **Multiplication**: The multiplication is done with the upper 32-bits of both operands set to 0. One operand carrying the watermark is chosen, its lower 32-bits are set to 0 and added to the result of the multiplication to transfer the watermark.
- **And**: One operand containing the watermark is chosen and the other operand is combined in an or-operation with the constant `0xFFFFFFFF00000000`, s.t. each bit in the upper 32-bits is 1, keeping all bits of the watermark intact in the and-operation.
- **Or**: The same as Addition.
- **Phi**: All operands need to contain the watermark to reliably propagate the watermark in any combination of control-flows. Therefore 3 cases have to be distinguished for the operands:
  - The operand is a usage cast of an expanded candidate, which can be directly used in the new Phi function.
  - The operand doesn't have a 64-bit version. A cast from 32-bit to 64-bit is inserted *after its definition*. It can't be inserted anywhere else since it has to dominate the new Phi function and Phi functions have to be at the top of each basic block. Since the 32-bit version dominates the old

Phi function, the inserted cast after the definition dominates the new Phi function, if it is inserted after the old.

- For any other value that is no Instruction, like Constants or Parameters, a cast is inserted in the Entry Basic Block since it dominates all other basic blocks and non-instruction values should be invariant to the location of the instruction inside a function.

- **Select**: Like for Phi-functions, all operands need to contain the watermark, the only difference is, that the insertion of the casts can be inserted directly before the new instruction again.

Of course, the list can be expanded to many more operations and the explained approaches are more easily detectable, since they follow common patterns (they produce many operations with the above-mentioned constants, that perhaps don't appear as frequently in unmodified programs). Nevertheless, those approaches insert fewer instructions than more sophisticated ones like vectorization instructions, leading to a lower performance impact and they are simple to implement.

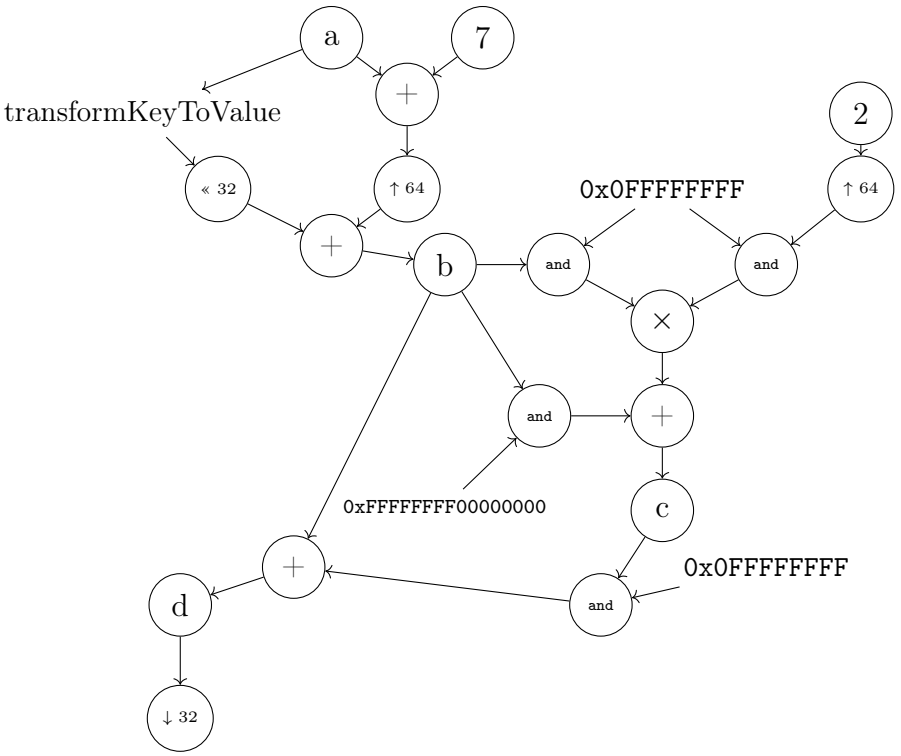


Figure 4.5: Visualization of the data flow graph after the insertion of the Register Expansion Watermark for the example from figure 4.2

The final transformation for the example from figure 4.2 can be seen in figure 4.5.

- From the expansion of b, the 64-bit cast (see figure 4.4) remains, while the 32-bit usage cast was deleted. The watermark was generated from the parameter a and added to b by the $transformKeyToValue$ procedure and a left-shift denoted by the « 32 node. b is the first variable that receives the watermark by construction it from the key in the parameter.

- For the multiplication the constant 2 was expanded to 64-bit and its upper bits set to 0. The upper bits of b are set to 0 as well. The normal multiplication is carried out in 64-bit, while the upper 32-bit should remain unaffected if not overflow happens. The upper 32-bits of b are selected by an `and` instruction and added to the result of the multiplication, transfering the watermark. If an overflow happened in the normal multiplication the watermark might be distorted.

- The higher 32-bits from c are set to 0 by an `and` instruction with a constant, s.t. the watermark in b is propagated to d by the addition with c.

- d is lowered back to 32-bit s.t. the correct type is returned.

The watermark can be found in the upper bytes of b, c, and d during execution. It can be argued that the inclusion introduces many new instructions, that could slow down the program and highlight the watermark in the binary. However, e.g. the down-cast instruction will be removed when the intermediate representation is compiled to x86 by using the 32-bit version of the register instead of the 64-bit version. Clang is in general capable of reducing the `and` operation with the constant `0x0FFFFFFFF` to `mov` instructions between register versions. So the compiler is capable of reducing and optimizing the newly introduced instructions.

In the following description of the insertion procedure, the creation of the new instruction that is capable of inheriting the watermark and the modification of the operands is abbreviated with the function `modifyInstruction()` since the procedure has been explained already.

`replaceUsage` is the procedure from algorithm 2 and `transformKeyToValue` from algorithm 3. Each operand of each patched instruction is either expanded to 64-bit or its 64-bit version is used directly. In the case where the instruction is a Phi-function or Select and the watermark has not been included in the operand, it has not only to be casted to 64-bit, but the watermark needs to be added to the upper bits, because for those instruction types all operands need to contain the watermark. When no data-flow connection to a parameter exists, we must introduce an artificial one that just generates the watermark from a random parameter and adds it.

Algorithm 4 Insertion of the Watermark

```
procedure INSERTWATERMARK(F : Function, I : Instruction)
    parent ← find candidate in operands
    if parent = ⊥ then
        param = find parameter of F in operands
        watermark ← transformKeyToValue(param)
        expanded ← I + (watermark « 32)
        replaceUsage(I, expanded, F)
    else
        newOperands ← ∅
        for all o : Operand ∈ operands(I) do
            if o has 64-bit version then
                o′ ← 64-bit version of o
            else
                if o is Instruction then
                    o′ ← cast(o, 64) after o
                else
                    o′ ← cast(o, 64) in entry block of F
                end if
                if I is a Phi-function or Select then
                    watermark ← transformKeyToValue(p)
                                        where p ∈ parameters(F)
                    o′ ← o′ + (watermark « 32)
                end if
            end if
            newOperands ← newOperands ∪ {o′}
        end for
        I′ ← modifyInstruction(I, newOperands, parent)
        replaceUsage(I, I′, F)
    end if
end procedure
```

The procedure is shown step by step for an additional example listed below in figure 4.6, which calculates the absolute difference of its parameters, to show the transformations in actual LLVM IR. The first step is identifying the candidates. In the first iteration, the variables 5 and 7 are selected since they have the correct instruction type and access the integer parameters. In the second iteration 9 is selected, because it accesses at least one other candidate (in this case 5 and 7). At first, each variable is expanded with casts to 64-bit and then cast back to 32-bit, the latter casts remain in the final program: `RegWatermarkCast`, `RegWatermarkCast1`, and `RegWatermarkCast2`). 5 and 7 don't contain previous candidates in their operands, so they first have to be cast to 64-bit (see `Cast1` and `Cast2`).

Then for each, random instructions are generated to mutate the watermark key contained in the first parameter. For 5, a multiplication, and for 7, a multiplication and division are inserted, leading to a different difference to the watermark value, which is added. While the multiplications can lead to an integer overflow, this is not a problem since it will be taken into account for the difference. Each time the generated watermark value is cast to 64-bit (See `Casted` and `Casted3`), shifted left by 32-bit, and added to the cast variable (See 16 and 9). The new Phi function is initially generated by casting the phi function to 64-byte and back to 32-byte (see `RegWatermarkCast`). After 5 and 7 were expanded to 9 and 16, their 32-bit versions (`RegWatermarkCast1` and `RegWatermarkCast2`) are inserted as operands in the phi function and the watermark can then be inserted into the phi function by replacing the 32-bit operation with a 64-bit that uses the 64-bit variants of its operands, i.e. 9 and 16. The original 32-bit calculation can then be deleted.

In this example the 32-bit versions `RegWatermarkCast1` and `RegWatermarkCast2` of the watermarked values 9 and 16 could be deleted since the Phi-function of `RegWatermark` only uses the watermarked values. Such optimizations are left to be carried out by the rest of the optimization pipeline e.g. by dead-code removal since in general they cannot be removed and may still be needed in instructions with unsupported instruction types, i.e. in variables that will not be watermarked. Although the code grows substantially, the optimizations of LLVM can reduce and tune the code. This can be seen in the execution times of the benchmarks 4.1.3: The benchmarks suffer a minimal performance toll from the watermark.

```
define i32 @a(i32 %0, i32 %1) {
  %3 = icmp sgt i32 %0, %1
  br i1 %3, label %4, label %6
4:
  %5 = sub nsw i32 %0, %1
  br label %8
6:
  %7 = sub nsw i32 %1, %0
  br label %8
8:
  %9 = phi i32 [%5, %4],[%7, %6]
  ret i32 %9
}
```

```
define i32 @a(i32 %0, i32 %1) {
  %3 = icmp sgt i32 %0, %1
  br i1 %3, label %4, label %10
4:
  %5 = sub nsw i32 %0, %1
  %Cast1 = zext i32 %5 to i64
  %6 = mul i32 %0, 7
  %7 = add i32 %6, -1029091587
  %Casted = zext i32 %7 to i64
  %8 = shl i64 %Casted, 32
  %9 = add i64 %Cast1, %8
  %RegWatermarkCast1 =
    trunc i64 %9 to i32
  br label %17
10:
  %11 = sub nsw i32 %1, %0
  %Cast2 = zext i32 %11 to i64
  %12 = mul i32 %0, 6
  %13 = sdiv i32 %12, 2
  %14 = add i32 %13, -1374056715
  %Casted3 = zext i32 %14 to i64
  %15 = shl i64 %Casted3, 32
  %16 = add i64 %Cast2, %15
  %RegWatermarkCast2 =
    trunc i64 %16 to i32
  br label %17
17:
  %RegWatermark = phi i64 [%9, %4],
                          [%16, %10]
  %RegWatermarkCast = trunc i64
          %RegWatermark to i32
  ret i32 %RegWatermarkCast
}
```

Figure 4.6: Example LLVM IR before and after the watermark insertion

4.1.2 Detection & Removal

```
int foo(int x){
 int d = x;
 if(d < 0) d *= -1;
 return d;
}
int main() {
 int x = getchar();
 foo(x);
}
```

Figure 4.7: Example of user input being directly passed to a function

For Detection of the watermark, a debugger that allows to view the register and stack contents, like `gdb` is necessary. Since the watermark only manifests itself when the watermark key is present in all 32-bit integer parameters one can trigger it either by choosing the correct user input - but this is only viable if the user input is parsed to a 32-bit integer and directly inserted as a function parameter in a patched function (e.g. see 4.7) - or by setting the function parameters with the debugger accordingly before the function is run (at the beginning of a function or before stepping into a call instruction).

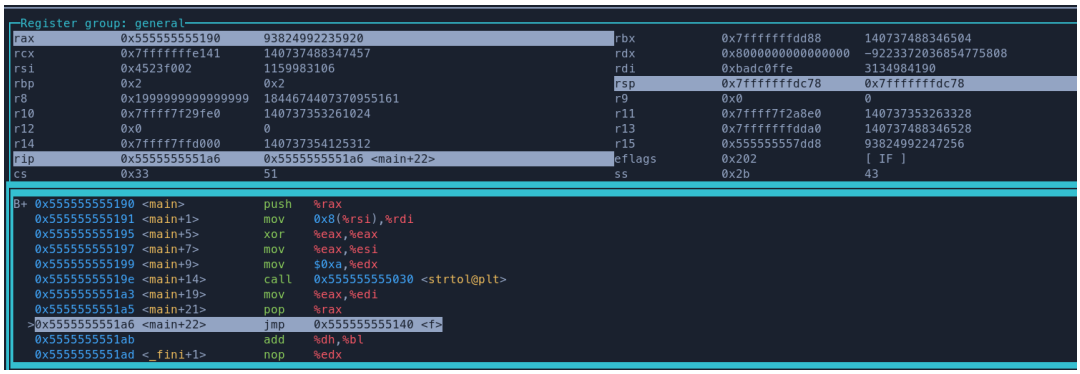


Figure 4.8: Example in `gdb`, the watermark key `0xbadc0ffe` was manually inserted in `rdi`

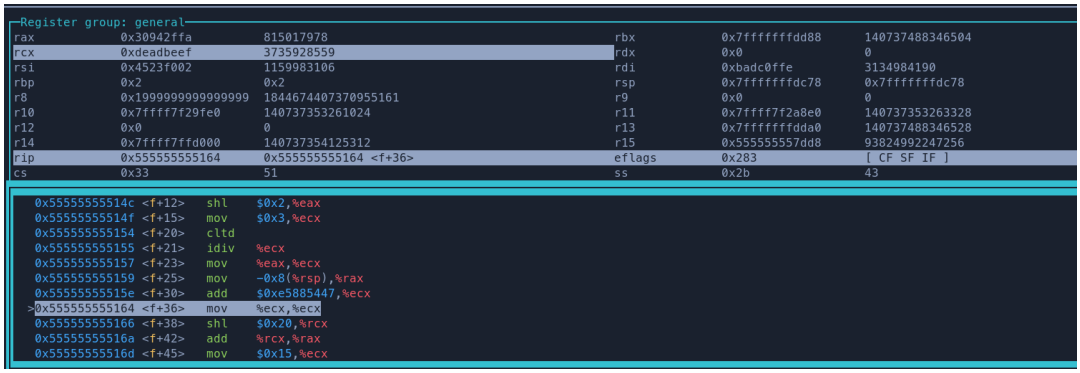


Figure 4.9: Example in `gdb`, the watermark value `0xdeadbeef` has been generated in `rcx`

The figures 4.8 and 4.9 show a program in the `gdb` debugger that has been watermarked with the key `0xbadc0ffe` and the value `0xdeadbeef`. In figure 4.8 the key was inserted in the parameter register `rdi` right before the call to the patched function `f` and figure 4.9 shows the first occurance of the generated watermark value in register `rcx` before it is left-shifted and added to other values.

Detecting the watermark without a debugger is very difficult since it is inserted in such a way that the program behavior must not change and that it manifests itself in the upper bits of a register, which is usually not visible to the user.

Since modifying parameters will lead to different return values or behavior in most functions, it is helpful to know which functions have been patched. The developer can document a list of patched functions during compilation and for detection set break points in `gdb`, set the parameters to the watermark key, and run the function instruction by instruction. The watermark will then appear occasionally in the upper bytes of the registers. Without the list, all parameters for function calls have to be patched with the watermark key.

With `gdb` scripting, this procedure is more or less automatable. After the program is started in `gdb`, it is possible to patch all parameters that contain 32-bit values (i.e. those which are smaller than `0x0000000100000000`) to the watermark key. After the patching, the upper 32-bits of all register values have to be checked per instruction to check if the watermark value is present. If it is, the execution can be stopped and the watermark was successfully detected.

However, the patching of the parameters will in many cases lead to different behavior like infinite-loops, different control-flow, or segmentation faults. If a function that was not patched with the watermark triggers such a problem, because of the altered parameters, it may lead to a program halt or control-flow that does not call patched functions that would have been called otherwise. Because of that, the automatic detection performs poorly and often does not detect watermarks that are detectable if the detection is done manually. Some program branches are only executed under specific conditions, if a watermark should be triggered in such a branch, fitting input has to be chosen as well.

Additionally stepping with `gdb` through every single instruction and checking the registers each time leads to very poor performance. An improvement would be to set a breakpoint in the main function and jump to it (which is possible since every program has a main function) and to prevent patching of parameters to extern functions, that can sometimes be detected by their name in `gdb`. Nevertheless, a complete automatic detection is not needed in most cases since mainly the developer of a program or library wants to detect their binary, so the list of patched functions is known prior. In those cases, manual detection will be most of the time more successful and efficient. Then it is even possible to transform the `gdb` script s.t. it takes the list of patched functions as input and sets break points for them, leading to a semi-automatic detection. If the

watermark is inserted in specific control-flow branches or the program expects specific dynamic input or user interaction, the script has at least to be modified to account for that.

The detection in byte code is difficult since the instructions have been interwoven with the functionality of the program. While in the intermediate representation, the watermark is still obvious, additional optimizations and transformation to assembly already provide a kind of obfuscation that can largely prevent detection of the watermark as described in [18](3).

However, analyzing statistics of the occurance of constants with an absolute value larger than 1024 (to eliminate constants like the initialization of counter variables etc.) or an increased usage of instructions generated by the watermark could give a hint if the binary contains the Register Expansion Watermark. Especially the random generated numbers and left-shifts that are used in the key-to-value transformation and in some inheritance procedures could be salient in the binary. An increased usage of the value 32 used in the left-shifts could also be a hint to the presence of the watermark. To test this hypothesis we calculated the percentile of left-shift instructions and the mentioned constant values in the binaries of the test suite where the watermark could have been inserted at least once, each compiled with the watermark and without:

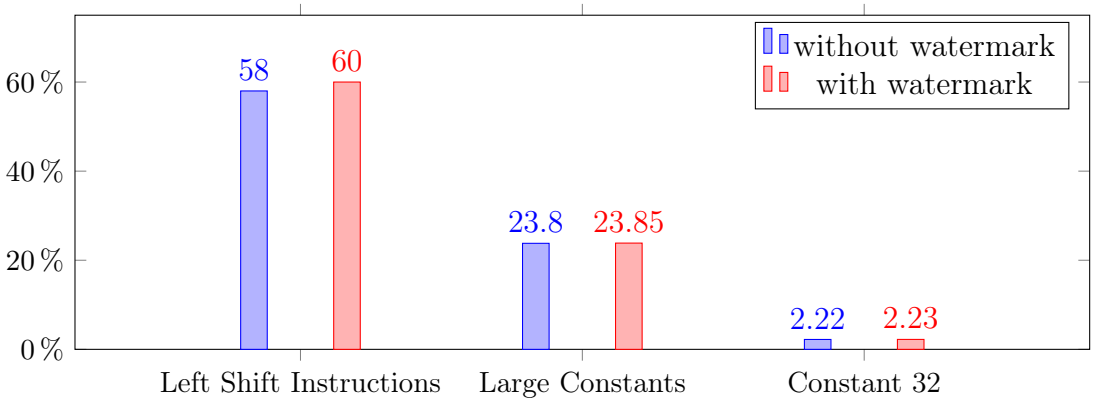


Figure 4.10: Percentile of generated left-shift instructions, constants with an absolute value larger than 1024 and the constant value 32 in the affected programs in the LLVM test-suite

While there is a measurable difference it is not enough to detect the watermark in binaries without prior knowledge. As already mentioned LLVM is capable of optimizing certain shifts or logical operations with bit-masks by using `mov` instructions. This may decrease the number of specific bit-masks and the constant 32 in the generated code of the watermark. Additionally, pointer arithmetic and calculations of stack-addresses can influence the occurance of large numbers.

To analyze the robustness and resilience of the watermark, each attack type described

by Christian Collberg and Clark Thomborson [3] is discussed. The removal (subtractive attack) of this watermark could be automated by analyzing for which register-writes or stack-entry-writes the upper 32-bit are used by other function calls, return values, or instructions with side effects and which are unused. While it is possible to distinguish between 32-bit and 64-bit values by looking at the used register types (32-bit or 64-bit versions) other problems arise. The return instruction `ret` assumes that the first returned value is in the `rax` register [5](4), which may contain watermark calculations in its upper bytes. From the instruction alone it is difficult to determine if the value should be 32-bit or 64-bit, so an analysis across function definitions and function calls is necessary (this can be seen in the example 4.6, where the compiled function `a` has an 8-byte write as the last writing-instruction to `rax` before termination, i.e. the x86 bytecode no longer contains `RegWatermarkCast`, but directly returns `RegWatermark`).
For those register-usages that have unused upper 32-bit another analysis is needed to determine which instructions only affect the unused upper bytes, those could then be deleted. The register can then be lowered back to its 32-bit version and the algorithm has to start again since the removal of instructions may cause other upper 32-bits of registers to become unused, so finally it is a fixpoint analysis and transformation.
Doing such transformations across function definitions and calls can become time-consuming for large programs and complicated to implement. The removal algorithm would need to consider many special cases of inlining and outlining to be resilient to optimizations and obfuscation of the watermark.

Compromising (distortive attack) just the watermark is difficult without the previously mentioned analysis of unused upper 32-bits since else the risk of compromising the used 64-bit values is high. In other words, a distortive attack carries in 64-bit programs too high risk of corrupting the original program.

With the analysis, reliable distortion is possible, however, then it would be arguably simple to just remove the watermark completely. Additive attacks with other Register Expansion Watermarks wouldn't have any effects since only 32-bit values are viable candidates to carry the watermark, but in an already modified program, all values carrying watermarks are 64-bit values. In summary, the watermark can be automatically removed and destroyed, but for this, complex algorithms and a full knowledge of the nature and insertion of the watermark are necessary.

4.1.3 Evaluation

To test the compilation and execution toll of the watermark the test-suite was compiled once with all optimization enabled (`-O3` flag) and again with all optimization disabled (`-O0` flag). The compilation from C to LLVM IR always has to include at least optimization level 1 since else clang marks every function with a `nonopt` attribute that forces *opt* to ignore optimization passes including the watermark pass. Each compilation is carried out twice once with watermark insertion and once without. In the chart in 4.11,

the compilation and execution times for the test-suite have been listed for comparison.

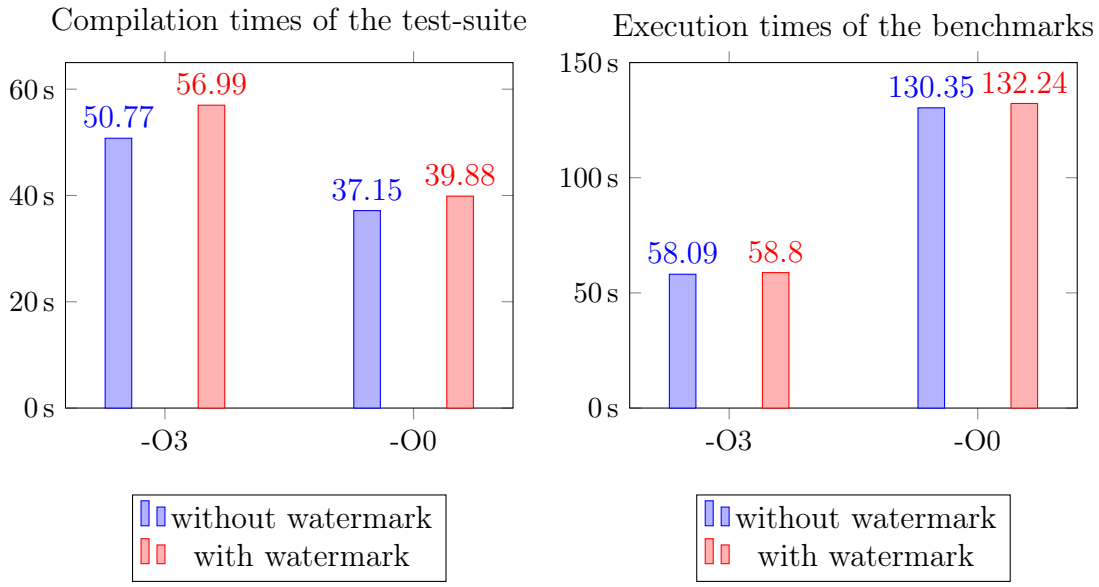


Figure 4.11: Compilation of the test-suite and execution of its benchmarks with and without the Register Expansion Watermark

The watermark was included in 33 / 93 benchmarks in a total of 401 functions so in roughly 35% of the programs. No higher level (e.g. O2 or O3) optimizations must be run on the programs before the watermark is inserted since aggressive function inlining or similar optimizations reduce the places where the watermark can be inserted heavily. The watermark can only be included in 9 / 93 benchmarks if the LLVM IR was optimized before the insertion. After the insertion, currently even aggressive optimizations don't interfere with the watermark and can be run. As already mentioned this may change with a better analysis from the compiler.

The benchmarks represent small, but time-consuming programs and often don't contain fitting functions, i.e. functions with 32-bit integer parameters and fitting operations. Regarding the mean number of lines of 76 per benchmark, each one can't be compared to a larger program or library that can contain millions of lines of code (the LLVM library alone containing llc, opt, etc. for example has 2601247 lines of code, not including clang, libc or other related projects). Additionally, as described in 4.1.1, many simplifications like a small selection of instruction types were taken that decreases the number of viable variables the watermark can be inserted into.

Therefore a study on a larger program collection is interesting. With a compiler dub for a modified clang version that includes the watermark, the GNU core utilities [6] were compiled. The watermark had been included successfully into 122 of 1348 C func-

tions in the source files, so roughly 9.1% (in most functions multiple times). It is worth mentioning that those sources mostly contain system and hardware interactions rather than arithmetic calculations, where the watermark would have more possibilities to be included, as was shown for the benchmarks.
LLVM [20] (without any additional projects like clang, just the base project) has been compiled as well and the watermark was included in 6690 of 126409 functions, so in roughly 5.2%.

It can be seen in figure 4.11 that the compilation time for the test-suite itself shows a measurable difference once the watermark is included, but relative to the compilation times without the watermark this difference is bearable, at least for static compilers. The execution of the benchmarks - i.e. the times the compiled programs with and without the watermark took to complete - shows nearly no measurable difference, meaning the performance toll of the watermark is also low.

Figure 4.12 shows that although measurable, the difference in binary sizes is low as well. While the watermark can insert many additional instructions, the fact that it was only inserted in a portion of the benchmarks and that not every function is viable for the insertion, its actual overhead should pose no problem in most cases.

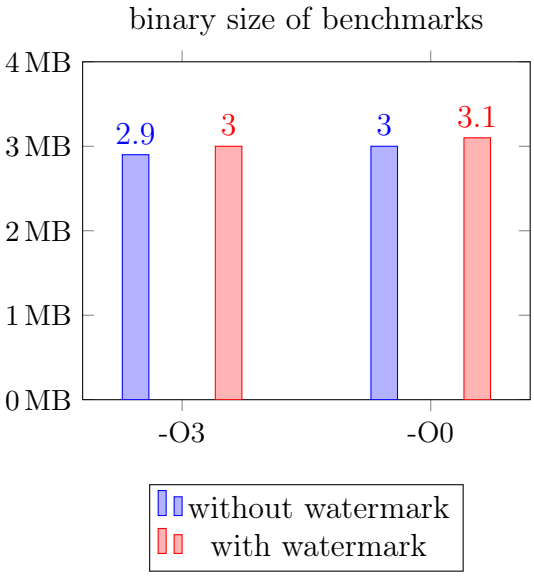


Figure 4.12: Size of the compiled benchmark binaries of the test-suite with and without the Register Expansion Watermark

Finally a study on a performance-intensive program that uses e.g. vector instructions would be interesting to determine if the watermark hinders the inclusion of vector instructions. Therefore a cache-optimized matrix multiplication program that implements the matrix multiplication in a separate function, which receives - aside from the arrays - the dimensions of the arrays as integer values, has been implemented. To include the watermark in the actual calculation, each multiplication from the parameter arrays is multiplied by the total size of the resulting array (since the watermark assumes that the key is in the array dimensions). We manually verified that vector instructions as well as the watermark have been conserved after compilation. The program was run 10 times, each with the watermark and without with randomly generated integer matrices of the size 2000 x 4000 and 4000 x 2000. The results can be seen in figure 4.13. While there is a certain performance difference, it is still moderate for such a "worst-case" scenario.

We did not measure the memory consumption since the watermark does not allocate

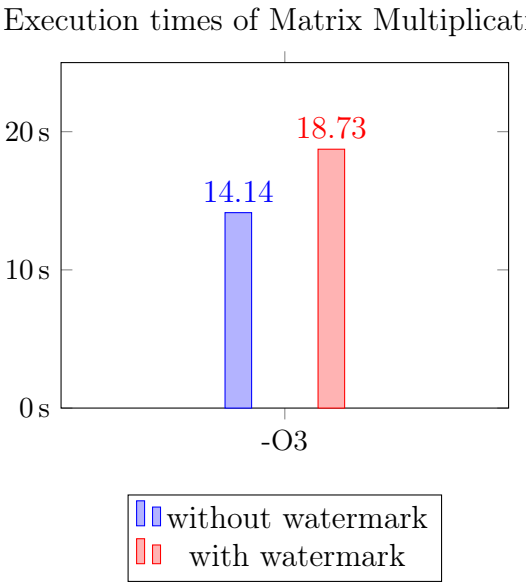


Figure 4.13: Execution time of Matrix Multiplication with and without the *Register Expansion Watermark*

dynamic memory. Depending on the function, the stack consumption may increase since the backend may assign slots for each expanded variable on the stack for spilling e.g. before function calls. However since only the variables are affected and each variable - if chosen as a candidate - doubles in size, the stack frame in the worst case doubles in size as well. In most cases, only a small fraction of the variables are chosen from a fraction of the functions (see inclusion in the GNU Core Utils and test-suite). This may affect the recursion depth for recursive functions but should have nearly no impact for most applications.

Overall the watermark effectiveness can be evaluated with the discussed aspects on the six criteria:

- Credibility: Since the modified programs generate the watermark value upon a "prompt" in the form of modified values inside the parameters the combination of both is credible. The probability that such a combination of exactly two values happens by chance is very low, yet the inserted watermark must be documented during the development of the software credibly since two integer values have - compared to e.g. a string with a company name - low data throughput and in any unmodified program it is possible to find parameter-value combinations that appear in the same way. During the evaluation of a program, it is important to make sure that:
  - the watermarks key-value pairs have been known before the investigation

of the program, s.t. it is ensured that they aren't picked to appear in the program by chance.

  - the watermark value manifests itself in multiple places if the key is present, which should be the case when the program has been compiled with the insertion procedure, as it was shown with the test-suite, the GNU core utilities, and LLVM. This makes the argumentation more credible, that the watermark key and value don't have been constructed to fit in specific places, but have been included during compilation.

- Robustness: Since the watermark is dynamic and interwoven with the program's instructions, it is difficult to remove. Nevertheless, we have shown in 4.1.2 that with enough effort it is possible to write an analysis and transformation pass that automatically removes the watermark and undoes its insertion without changing the behavior of the program since the watermark is not relevant to the behavior.

- Transparency: Complete automatic detection, although possible, is very difficult and seldomly effective for the watermark. Nevertheless, manual or semi-automatic detection is easy and effective if the functions in which the watermark had been included are known during the investigation, making it hard for any third party to trigger the watermark, but easy for the developer if they document the list of affected functions.

- Resilience: Compromising the watermark without compromising used 64-bit values of the program is arguably just as hard as removing it, however, just as well possible to automate.

- Throughput: The watermark itself is capable of storing a 32-bit value, taking the watermark key as an information-source into account as well the total information throughput allows 2 bytes, which is not much storage for identifying information, so the key-value pair has to be chosen carefully to allow a credible detection.
  Theoretically, it is possible to allow multiple key-value pairs, e.g. one per function, to be included in each program, allowing a throughput size just depending on the size of the program, however, this causes some problems as well. The detection of the watermark would be harder, and arguably only possible with the list of functions and their corresponding watermark key-value pairs and the number of includable watermarks would depend on the number and types of the present functions in a program.

- Runtime & Memory consumption: since the watermark does not allocate dynamic memory and at most doubles the stack frame per function, the memory consumption is neglectable. The performance toll is - as has been shown with the benchmarks of the test-suite - very low. The same goes for binary sizes. While the watermark would have a larger impact on runtime and binary sizes for special

cases where it would be included in far more places, especially in values which's calculations are more performance-intensive, such cases did not appear in our test set, due to the small viable set of functions that can produce it.

Future work about the Register Expansion Watermark might investigate the following aspects, that were not directly studied in our work due to time limitations and being beyond the "proof-of-concept" nature of the topic:

- Inclusion of additional instruction types and measurements of the increase of watermark insertions.
- Implementation of multiple watermark key-value pairs in one program for higher throughput, with a study about detectability.
- Implementation of an automatic removal of the watermark with performance tests.

4.2 SemaCall Watermark

The *Register Expansion Watermark* showed 2 major disadvantages:

- Removal: it is possible to write an automatic removal optimization for the watermark.
- Throughput: the basic version is only capable of an information-throughput of 8 bytes.

While especially the on-spot generation of the watermark plays well into its dynamic nature since it makes it much harder to detect it and does not include literal evidence of the watermark. A simple way to increase throughput arbitrarily is to widen the target values to arrays, especially strings since they allow storage of human-readable text. Storing them in padding limits the throughput and will arguably be seldom possible for an adequate number of characters. Instead, the generation of such watermarks can be interwoven in the target programs by introducing side effects, the most obvious one would be a simple print statement on the standard output. Function calls that use strings in their parameters that may or may not contain user interactions, but guarantee that the variable contains a zero-terminated string can be extended with a condition that checks if the watermark key is present and triggers the side effect with the value. This can lead to poor performance since without symbolic execution or Just in Time compilation information about the overhead of the function or the performance-criticality of the function call, it is hard to estimate the cost of the watermark. Instead, the viable functions can be limited to a set of predefined functions, that are designed to perform system and therefore user interactions: a selection of the standard library of the programming language, in this case, `libc`. Although functions dealing with text are preferable since they allow to deal with arbitrarily large texts, other functions, which work on other dynamic user input or system information can be used too. E.g. the system's datetime, operation system version, free space on a harddrive, etc. could be used to decode a watermark as well, the advantages and disadvantages depend on the type of information. Knowing the semantics and overhead of the viable functions allows one to write customized watermark insertions. We call this the SemaCall (**Sema**ntically-Known Function-**Call**) Watermark since it is based on the known semantics of function calls and their dynamic information.

Per function-call, a transformation can be written that detects the watermark key and generates the watermark value similar to the randomized value transformations we used in the Register Expansion Watermark. E.g. a function includes a function-call to the placeholder function `a`, for which such a transformation exists, with a string as a parameter, as can be seen in 4.14. It would be possible to write a transformation from the dynamic input to the watermark if the watermark is again treated as a key-value pair of strings with the same length, s.t. a per-character value transformation becomes

```
void some_function() {
 char* str;
 // ...
 char* x = a(str);
}
```

```
void some_function() {
 char* str;
 // ...
 long random = 0xdeadbabe;
 if (hash(str) == random) {
  char* watermark = transform(str);
  printf(watermark);
 }
 char* x = a(str);
}
```

Figure 4.14: pattern of a patchable function call in the SemaCall Watermark

possible. There are several options on how to store or manifest the watermark, e.g. exploit padding in structures (data structure watermark) or using existing calls to `write` or `printf` (easter egg watermark), we will use a dynamic easter-egg approach for simplicity. Collberg et al. describe easter-eggs as a "piece of code that gets activated for a highly unusual input to the application" [3], they state that the main disadvantage of an easter-egg watermark lies in its detectability because it is apparent from the assembly how to trigger the watermark and what its effects will be, especially if it is not very subtle. Instead, we will use hashing and transformation procedures from the *Register Expansion Watermark* to trigger and generate the watermark, which complicate detection and reconstruction of the right prompt. To test for the activation of the easter-egg a hash value is generated from the variable containing the dynamic information, which can be tested for an expected value. This again removes the necessity to include literal evidence of the watermark. If the easter-egg is triggered, it is assumed that the variable contains the watermark key and the per-character value transformation then creates the watermark value which can be printed afterward.

4.2.1 Implementation

While it would be possible to write a generic transformation for all strings (for something similar, see the *Sidedata Watermark* from section 4.3), we can use different implementations for different functions to better hide the watermark or remove overhead for the hash generation. This allows the inclusion of other non-string system information and avoids the problem that from the LLVM-IR it is not apparent if a pointer points to string data since strings are stored in 1-byte integer pointers (just as in C with `char`). If a function works on strings, it can be assumed, that the parameter passed to it is a zero-terminated string. The selection of functions to include depends on multiple parameters, but mostly on the relative overhead of the watermark to the function call. For functions that need to use system calls, access devices or device drivers, or even read user input, the overhead

introduced by the watermark should be neglectable for a reasonable throughput size. For other functions that work as helper functions like `strlen` or `atoi` and do not need to access resources outside of the process, the overhead of generating the hash value for the watermark might surpass the time the function takes to complete. This would lead to rather poor performance for cases where calls to such functions happen inside of large loops or even recursive functions.

To test this hypothesis we implement a transformation for `atoi`, `atol` and `strtol` (referred to as the `atoi` implementation). Additionally, a set of functions that may access system resources has been chosen:

- `gets`, `fgets`, `getline` (referred to as the `gets` implementation)
- `time`, `gettimeofday` (referred to as the `time` implementation)

These are heavily used functions that commonly deal with user input on the terminal, in files, servers on the network, or in the case of `time` dynamic system information, allowing the insertion of the watermark key in various ways. While `atoi`, `atol`, and `strtol` don't directly connect to user input, they can be used to check and parse text input that was inserted by the user, offering the possibility to check for the watermark after it has been read by another function. `time` allows to decode a specific datetime as the watermark key, that can be triggered if the datetime is manually set to that date. This is most effective if the time is in the past since this reduces the cases where the watermark is triggered erroneously, however, the watermark key and value are limited to a fixed throughput size (in its implementation, we will develop a technique that only limits the key and allows an arbitrary large throughput for the value).

General Implementation

The implementation has to detect calls to the selected functions. This is in most cases not as simple as it sounds because in the optimization stage of `llvm`, the origin of the implementation of a function to `libc` is not yet known because library calls to `libc` are usually linked dynamically, meaning this information is only available in and after the stage of the linker. To simplify this each implementation accepts functions that have the same name and parameter types as the selected function.

The original basic block with the function call is then split into three. The original block is extended by the generation of the hash value either in place (possible for functions that generate integers) or by calling a hash function that can be reused. The hash function transforms the expected key to a value that is randomly chosen by the transformation. The last instruction of the block - which is according to the SSA-form a branch instruction - is replaced by a conditional jump checking if the hash is the expected random value. The original jump instruction is placed as the only instruction into another basic block, the *jump-block*. For this to work, all phi instructions referencing the original

block have to be adapted, s.t. the *jump-block* replaces the original block functionally. If the condition fails, the new branch instruction jumps to the *jump-block*, but if it holds it jumps to another new basic block, the *easter-block*. The *easter-block* contains transformation instructions that transform the information in the variable holding the key to the watermark value (in cases of strings each character is transformed). The transformed value is then printed.

This transformation of the control-flow-graph is shown for the example in 4.15, keep in mind, that in this example the basic block has an unconditional branch as the last instruction, so construction of the *jump-block* would not be necessary and can be removed by an optimization. However, this is generally not the case for conditional branching, where the condition and branch could not have been included in the branch instruction of the new B1 basic block. The example only shows a small fraction of a function and mixed C-code and LLVM-IR for clarity and brevity. The concrete adaptions on this

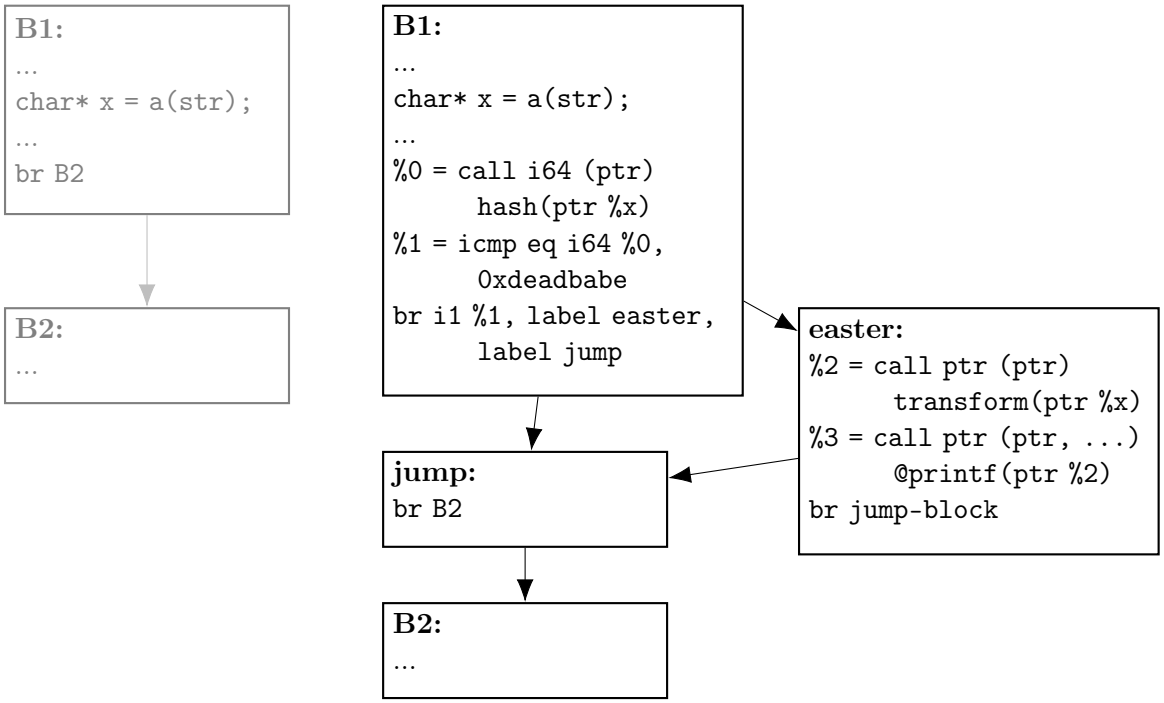


Figure 4.15: CFG Transformation for the SemaCall Watermark

general structure are explained in detail per function and implementation.

Character Transformation

A common procedure all of the selected functions, except for `time`, have in common, is the per-character transformation of the watermark key (or whatever variable is expected to hold the key, after the hash value matched) to the value.

One implementation revisits the key-value transformations performed in the *Register Expansion Watermark* 3, but per character. For each character i in the watermark key, random instructions like multiplication, addition, and division are generated with compile-time random values, that perform a hash transformation (again at least one modulo operation needs to be added) to a random value.

The difference to the expected value, i.e. the difference to the character of the watermark value at the same position i, is added afterward. This eliminates the need for literal inclusion of the watermark key and value, while still transforming the one to the other. However, a per-character transformation yields many instructions, which all do a similar task (transforming characters of a string into a new string) and are salient in the assembly code.

Another implementation would be to construct a mathematical function that yields the watermark value for each character, if the key is inserted. This would have the advantage, that the function can be evaluated per character in a loop and only has to be generated once, making it harder to detect this generation. Additionally inlining and loop-optimizations like vectorization could further obfuscate the watermark and improve its performance. Since the characters in the watermark key are not unique, they can't be mapped directly onto the x axis and the additional information of their position is required, i.e. a function

$$f_i \in x \to y,\ \forall x_i \in x.f_i(x_i) = y_i \in y \text{ with } i \in \{1, ..., |x|\} \text{ and } |x| = |y| \tag{4.1}$$

has to be found, where x is the watermark key and y is the watermark value. The positional encoding of the characters in x at position i can be achieved by defining:

$$f_i(a) = f(a + i \cdot 256) \tag{4.2}$$

where 256 represents the upper-bound of possible values for each character. A function definition of a function that meets a certain set of points - in this case the pairs of encoded character values of the watermark key and the corresponding characters of the watermark value - can be obtained by using the Lagrange Interpolating Polynomial explained and proofed in James F. Eppersons "An Introduction to Numerical Methods and Analysis" [4]:

$$f(a) = \sum_{i=0}^{n} y_i \cdot \left(\prod_{j=0, i \neq j}^{n} \frac{a - x_j}{x_i - x_j} \right) \tag{4.3}$$

Where $n = |x| = |y|$. That kind of function could be directly compiled to LLVM-IR and used to generate the per-character transformation of each character a in the input string. However, this comes with caveats:

- The literal evidence of the characters of the key x_i and - even worse - the value y_i are now hardcoded. While the constant-folding optimizations can fold and remove

the evidence of the characters in the fraction, tests on these kinds of functions concluded that LLVM was not capable of removing the evidence of y_i because this requires changes to the overall formula.

- Transforming one string now is in $\mathcal{O}(n^3)$ (each character transformation from equation 4.3 is in $\mathcal{O}(n^2)$), while the simple per-character transformation with hashing, described earlier, is in $\mathcal{O}(n)$. However this is not a big problem, since the easter-egg should not be triggered often.

Both problems could be solved or improved by simplifying the formula to a real polynomial of the form:

$$f(x) = \sum_{i=0}^{n-1} c_i \cdot x^i \tag{4.4}$$

Which is in $\mathcal{O}(n)$, so the complete transformation is in $\mathcal{O}(n^2)$. The Lagrange polynomial could be parsed into an Abstract Syntax Tree (AST), and reshaped by predefined transformations until it corresponds to the form of equation 4.4. An abstract pseudo-implementation of such a transformation could look like this:

procedure POLYNOMIALTOPOLYNOM(P)
  $changed \leftarrow \top$
  while $changed$ **do**
    $changed \leftarrow \bot$
    $P, changed \leftarrow multiplyTerms(P)$
    $P, changed \leftarrow constantFolding(P)$
    $P, changed \leftarrow flatten(P)$
    $P, changed \leftarrow group(P)$
  end while
return P
end procedure

polynomialToPolynom transforms the constructed Lagrange Interpolating Polynomial into a polynomial of the form described in the equation 4.4, by exhaustive application of a set of transformations. Each transformation corresponds to an arithmetic rule, the application of them together converges if P is in the correct form. Each returns the modified polynomial and true, if the rule applies, else they return the original polynomial and false:

- $multiplyTerms(P)$ searches for products of terms and multiplies the terms with each other. If the terms are divisions, *multiplyTerms* is applied recursively to both the numerator and denominator. If the terms are sums, all combinations of multiplications between the operands of the sum-terms are formed into products and the products combined into a new sum operation.

- $constantFolding(P)$ Finds operations containing multiple constant values that can be precomputed.

- $flatten(P)$ Finds operations with operands that have the same operation type (e.g. Sums of Sums, Products of Products), and adds the operands of the operand to the original operation and removed the operand.

- *group*(P) Finds sums that contain product terms with the variable, groups those terms by the number of variable occurrences they contain, and sums them up.

E.g. the transformation of the key string "test12345" to the value string "watermark" could be represented by this polynomial:

$$\begin{aligned} f(x) = 680.9137175331614 &+ (x \cdot -8.915436474640368) + (x^2 \cdot 0.0469304869604419) \\ &+ (x^3 \cdot -0.00011998055910209616) + (x^4 \cdot 1.7049268521109171 \cdot 10^{-07}) \\ &+ (x^5 \cdot -1.41602130716186 \cdot 10^{-10}) + (x^6 \cdot 6.83205573980654 \cdot 10^{-14}) \\ &+ (x^7 \cdot -1.7730434157274152 \cdot 10^{-17}) + (x^8 \cdot 1.9130301488006625 \cdot 10^{-21}) \end{aligned} \quad (4.5)$$

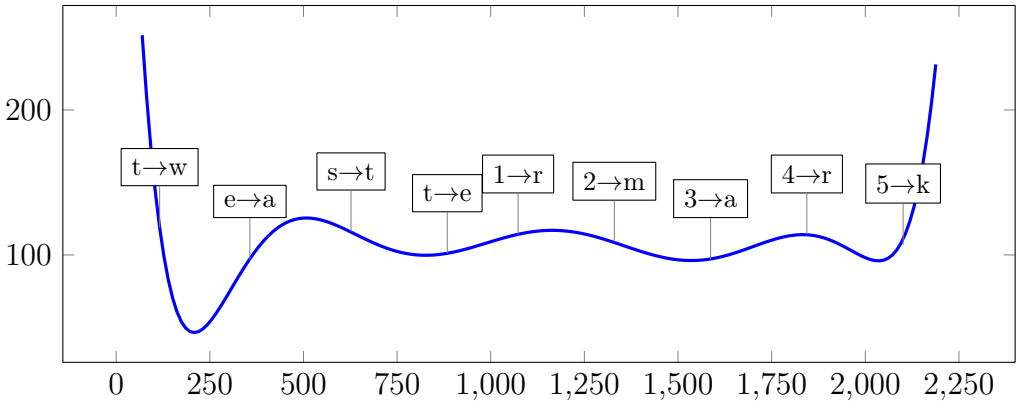


Figure 4.16: Plotted polynomial transforming the string "test12345" to "watermark"

As can be seen, the polynomial contains no longer literal evidence of the watermark value and keys and is compacted into 23 instructions (assuming there is a power instruction). This however, introduces new caveats:

- Already for the example above with strings of length 9, we get a polynomial of order 8, in fact, due to the nature of the Lagrange Interpolating Polynomial the order will always be $n - 1$. The coefficients go as low as $1.91 \cdot 10^{-21}$. Both will inevitably lead to rounding errors for the 64-bit floating point operands leading to "noise" and therefore errors in the watermark generation. Those rounding errors, of course, occur already in this small example, but are smaller than 1 and rounding the value to the next integer prevents small errors from influencing the result.

- The performance impact of the transformation to the polynomial is exponential, which makes its inclusion in a compiler toolchain unfeasible.

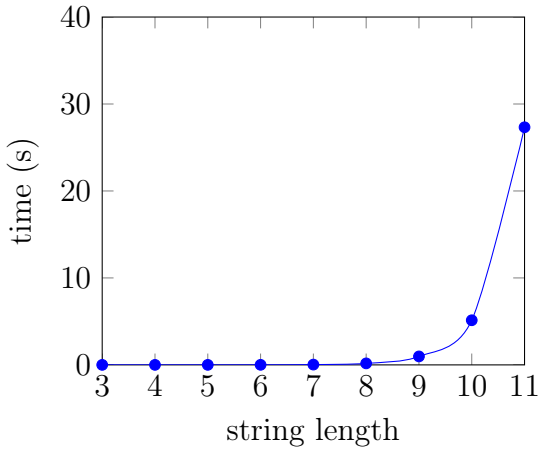


Figure 4.17: Time per string length for the construction of the key-value transformation polynom

This is because of the *multiplyTerms* transformation, which generates exponentially more terms by multiplying the operands of sums with each other. The graph 4.17 shows the average runtime of 5 samples per data point of the polynomial generation for different string lengths. Even though the polynomial has to be constructed only once per watermark key-value pair, this is an unacceptable time penalty in a compiler system.

The performance problem can be tackled by instead of rearranging the Lagrange Interpolating Polynomial to a polynom, directly constructing the polynomial. If the polynomial should be in the form:

$$p(x) = c_0 \cdot x^{n-1} + c_1 \cdot x^{n-2} + \cdots + c_{n-1} \cdot x^0$$
$$\text{with } p(x_0) = y_0,\ p(x_1) = y_1,\ \ldots,\ p(x_{n-1}) = y_{n-1} \quad (4.6)$$

the calculation of $c_0, c_1, \ldots$ can be done by solving

$$\underbrace{\begin{pmatrix} x_0^{n-1} & x_0^{n-2} & \ldots & x_0^0 \\ x_1^{n-1} & x_1^{n-2} & \ldots & x_1^0 \\ \vdots & \vdots & \vdots & \vdots \\ x_{n-1}^{n-1} & x_{n-1}^{n-2} & \ldots & x_{n-1}^0 \end{pmatrix}}_{X} \cdot \underbrace{\begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_{n-1} \end{pmatrix}}_{C} = \underbrace{\begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{pmatrix}}_{Y} \quad (4.7)$$

This can be solved by calculating $C = X^{-1} \cdot Y$. X is a Vandermonde matrix and its inversion is possible since, according to Turner et al., Vandermonde matrices are invertible [24]. Constructing this polynomial only takes a few milliseconds even for large strings with over 100 characters and the final polynomial is the same as the rearranged Lagrange Interpolating Polynomial, therefore is subject to the same numerical instabilities. In tests the first instabilities where the wrong rounded integer is generated (i.e. the wrong output character) occur starting with 17 characters, which limits the throughput to this number of characters. While optimizations like taking the *ith*-root of the coefficient for x^i and including them in the basis of the power operation, could improve the numerical stability, the improvements would only be marginal and just increase the

upper limit instead of removing it.

Therefore the simple per-character hash transformation is chosen for the string transformation of the watermark key to the value for the patched libc calls, which doesn't limit the throughput. However, in practice, it might be better to dynamically decide which of both transformations to use depending on the size of the watermark.

`atoi`, `atol` and `strtol`

These functions parse the first portion of a string to an integer value. While `atoi` and `atol` work like `strtol` with a null pointer for the end pointer and base 10 [7], they are not capable of detecting errors, unlike `strtol` [12].

The insertion of the watermark is mainly the same since all accept the provided string as their first argument and return either a 32-bit or 64-bit integer with the result. This task is purely arithmetic and does not need system calls or communication with devices or drivers, which makes these functions comparably lightweight. They are nevertheless included to investigate the performance toll of the watermark for such cases.

The most interesting optimization of these functions is, that they return integer values, which can be used to generate the hash value instead of the complete string, suspected to make the hashing faster. With this procedure, the hash can be generated in place in the basic block (without a function call) since no loops are required to generate a hash from an integer. This is done in the same randomized way in which we transformed an integer to another in the *Register Expansion Watermark* (see section 3), by using a random integer as the target hash value, i.e. the hash generation is generated in a way, to exactly transform the result of `atoi`, `atol` or `strtol` on the watermark key, to a random value generated beforehand, which is then used to check if the hash matches it, i.e. is the same value as the watermark key would produce with the hash generation.

During the watermark inclusion the return value of the `atoi`, `atol` or `strtol` is taken and applied to random multiplication, addition, modulo and division instructions with random values. The watermark key is passed through the function as well, and the effects of the random instructions is applied to its result during the inclusion. At least one modulo operation must be included. A random integer value is chosen and an addition instruction that adds the difference between the transformed watermark key to the transformed result of the function call generated, this corresponds to the algorithm 3, except that not the watermark value is used but the random integer, which now represents the hash value that triggers the easter-egg.

The generated hash can then be tested if it corresponds to the random value, if the original string was the watermark key, it does so by design. A branching instruction is generated that jumps to the *easter-block* if the hash matches, else normal execution is continued. However, a correct hash does not guarantee that the string contains the watermark key. A wrong detection in a too-short string can even lead to segmenta-

tion faults when generating the watermark value, therefore this can be prevented with multiple, combinable strategies:

- Choosing a watermark with a unique, hard-to-guess number in front. A watermark key like e.g. "Hello World" is bad because `atoi` would return 0. There are many cases where this watermark could be triggered falsely. Values like "9327502387secret" are far less likely to be triggered by chance. It has to be acknowledged that not the string length, but the length and uniqueness in the context of the program of the first occurring number determines the chance of an erroneous triggering.

- Additionally checking for the string length. This is quite useful because it prevents segfaults in an erroneous triggering. While it does introduce overhead, the check can be placed after the matching of the hash value, reducing the overhead to the cases where the watermark is triggered.

- Checking the complete string for the watermark. This is not advisable since it reintroduces the necessity of including literal evidence of the watermark, however, it is the only way to definitely prevent erroneous triggering.

The watermark value can be generated in the *easter-block* by the per-character hash transformation described earlier. This generation has to be hardcoded and can't be simply done in a loop since each character requires a different transformation. However, hardcoding the transformations shouldn't be a big problem if one applies further optimizations and obfuscation techniques to the generated code afterward, which already disguise and transform this generation, making it harder for a human to detect it.

The final control-flow-graph transformation is shown in 4.18, similar to the one in 4.15 with an additional test for the string length, mainly to show the adaption for these types of functions. While the check for the string length could have been included in the basic block B1, introducing a new one enables short-circuit evaluation i.e. it only calculates the string length if the hash matches (in the example 9).

The insertion in the following example program 4.19 would trigger the easter-egg for matching strings, but only produce the correct watermark for the correct key, the used watermark key was "12345test" and the value "watermark". While the input "12345abcd" does trigger the watermark because the input to the hash is the result of `atoi`, which returns 12345 for this string as well as for the watermark key, the watermark value printed in the *easter-block* is different because the transformation procedure generates different characters for "abcd" than it would for "test". "12345a" does not trigger it because it does not have the correct length.

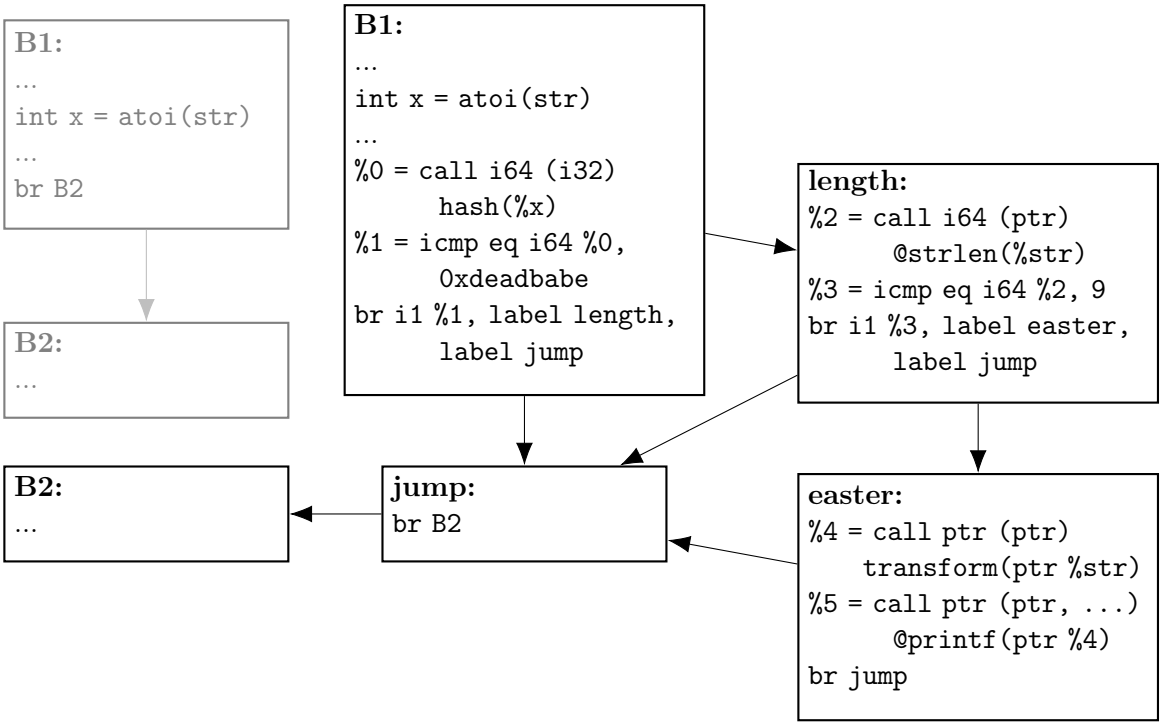


Figure 4.18: CFG Transformation for the SemaCall Watermark for `atoi`, `atol` and `strtol` with a check for string length

```
#include <stdlib.h>
#include <stdio.h>
int main(int argc, char** argv) {
  int arg1 = atoi(argv[1]);
  printf("%d\n", arg1);
  return arg1;
}
```

```
$ ./a.out 55abc
55
$ ./a.out 12345test
12345
watermark
$ ./a.out 12345abcd
12345
water+Obc
$ ./a.out 12345a
12345
```

Figure 4.19: Example C program, compiled to include the atoi-watermark and example inputs triggering the easter-egg

`fgets`, `gets`, `getline`

According to the Linux manual pages [10] `gets` is deprecated and should never be used because "it is impossible to tell without knowing the data in advance how many characters `gets()` will read". ISO C11 even removes the function from libc. Therefore it is

only included because of its similarities to `fgets` [8], which is much safer since it implements a character limit of the read bytes. Both `fgets` and `gets` receive - according to the Linux manual pages - a pointer to memory to store the result in, as their first argument, while `getline` [9] receives a pointer to such a pointer because the memory does not need to be allocated: if the underlying pointer is a null pointer, the needed memory is allocated by the function and a pointer to that memory is stored in the first parameter. We can therefore reuse the implementation of the watermark insertion in the `fgets` and `gets` calls for `getline`, as long as we dereference the argument pointer before checking the hash and generating the watermark.

An advantage of `getline` is, that it returns the size of the read string, which can be reused for checking for the correct string length as done in figure 4.18 by the *length-block*, reducing the overhead further because the call to `strtol` becomes redundant and can be removed.

Since those functions only handle strings (and additionally, at most their length and buffer limit), the hash function has to be computed from the final string these functions read. For the `atoi` implementation, the parsed integer could be taken to produce an in-place hash transformation, tailored to compute a specific value if the watermark key was inserted. For strings, a hash function regarding all characters of the string has to be chosen.

M.V. Ramakrishna and Justin Zobel propose a generic form of string hashing functions - shown by algorithm 5 - and discuss the performance of several implementations of the placeholder functions.

Especially because of its good uniformity, minimizing the number of erroneous triggers of the watermark, the proposed *shift-add-xor* hashing-function fits our use-case. It is defined by the following implementation [23]:

$$\begin{aligned} init(v) &= v \\ step(i,h,c) &= h \oplus (\mathcal{L}_L(h) + \mathcal{R}_R(h) + c) \\ final(h,v) &= h||T \end{aligned}$$

Where $\oplus$ represents exclusive bitwise or, $\mathcal{L}_b$ a left-shift and $\mathcal{R}_b$ a right-shift by b bytes and $||$ modulo. They chose $L = 5$ and $R = 2$. T is the table size, which is neglectable in the application case of recognizing the watermark key since overflows are no problem and there is no range in which the hash has to fit into.

Algorithm 5 Generic form of hash functions

procedure HASH(s, v)
  $h_0 \leftarrow init(v)$
  for each c_i in s **do**
    $h_i \leftarrow step(i, h_{i-1}, c_i)$
  end for
return $final(h_{|s|}, v)$
end procedure

Proposed by M.V. Ramakrishna and Justin Zobel [23], where s is the string, h_i the intermediate hash for iteration i and v a seed

Using a randomized seed would break the recognizability of the watermark key. Therefore the following adaption is used to implement this hash function for the usecase:

$$
\begin{aligned}
init(v) &= 7 \text{ (an arbitrary prime number)} \\
step(i,h,c) &= h \oplus (\mathcal{L}_5(h) + \mathcal{R}_2(h) + c) \\
final(h,v) &= h \text{ (no modulo since overflows are allowed)}
\end{aligned}
$$

The representation of this hash function in LLVM-IR is generated and inserted in each affected module. This reduces the final overhead of the watermark on the size of the program since multiple patches in one module can reuse the hash function. The returned integer still has to be subjected to randomly chosen instructions that constitute a hash transformation as described in Methodology (see chapter 3), to enable the uniqueness of the tested value per insertion point and the Non-Reconstructability. Alternatively, the initial value could be chosen randomly for each module, but this would mean that multiple insertion points in a module use the same constant to detect the key, which would again simplify the removal of the watermark, or each insertion point would need a individual hash-function, which would increase the binary size.

The same per-character hash transformation of the watermark key to the watermark value is chosen as for `atoi` etc. Only for `getline`, the check for the string length can be simplified since the number of read bytes is returned by the function. The final transformations are shown in Figure 4.22 for `gets` and `fgets` and in Figure 4.21 for `getline`.

Figure 4.20 shows the effect of the watermark in practice. A simple program that reads from the standard input stream with the `fgets` instruction and prints the result to the standard output stream is compiled with the watermark key "12345test" and the watermark value "watermark". For the input "12345word", the hash value does not match, so the program runs normaly. For the right input, i.e. the watermark key "12345test", the hash matches, the easter-egg is triggered, the watermark is computed from the key and printed on the standard output. Afterwards normal control-flow continues (in this case only the termination of the program, since it has no branches).

```
#include <stdio.h>
int main() {
        char input[1024];
        fgets(input, 1024, stdin);
        printf("%s\n", input);
}
```

```
$ ./a.out 12345word
12345word
$ ./a.out 12345test
12345test
watermark
```

Figure 4.20: Example C program, compiled to include the fgets-watermark with the key "12345test" and value "watermark" and example inputs triggering the easter-egg

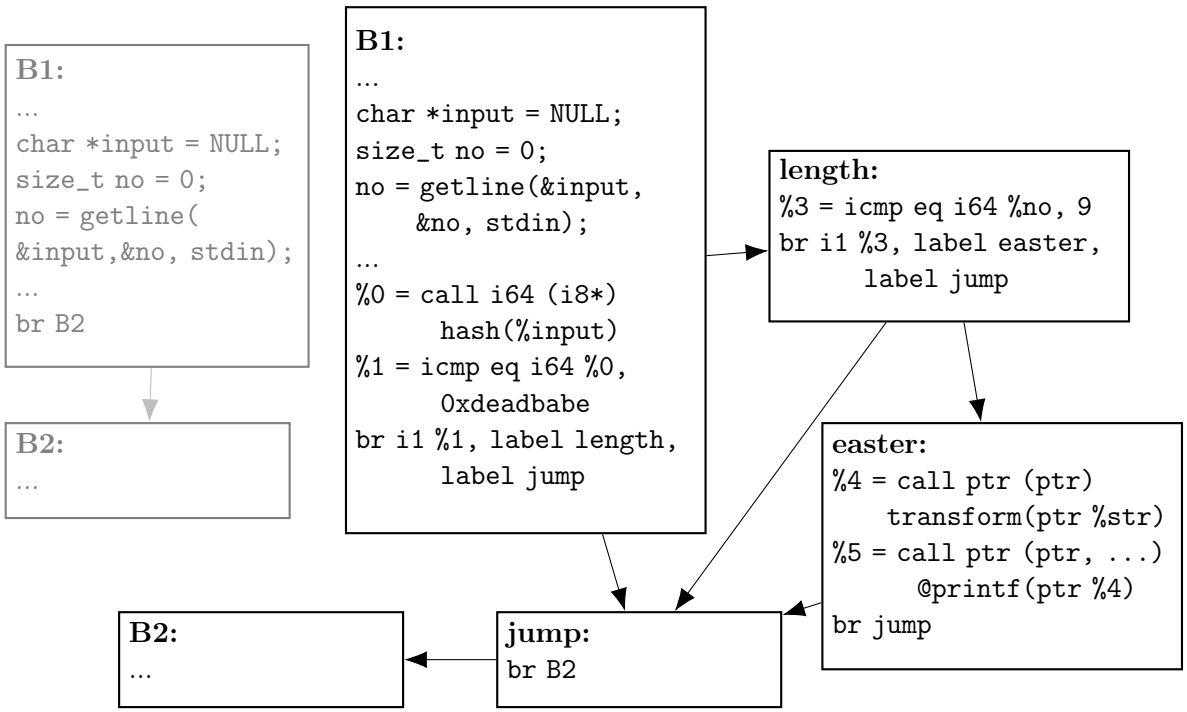


Figure 4.21: CFG Transformation for the SemaCall Watermark for `getline` with a check for string length

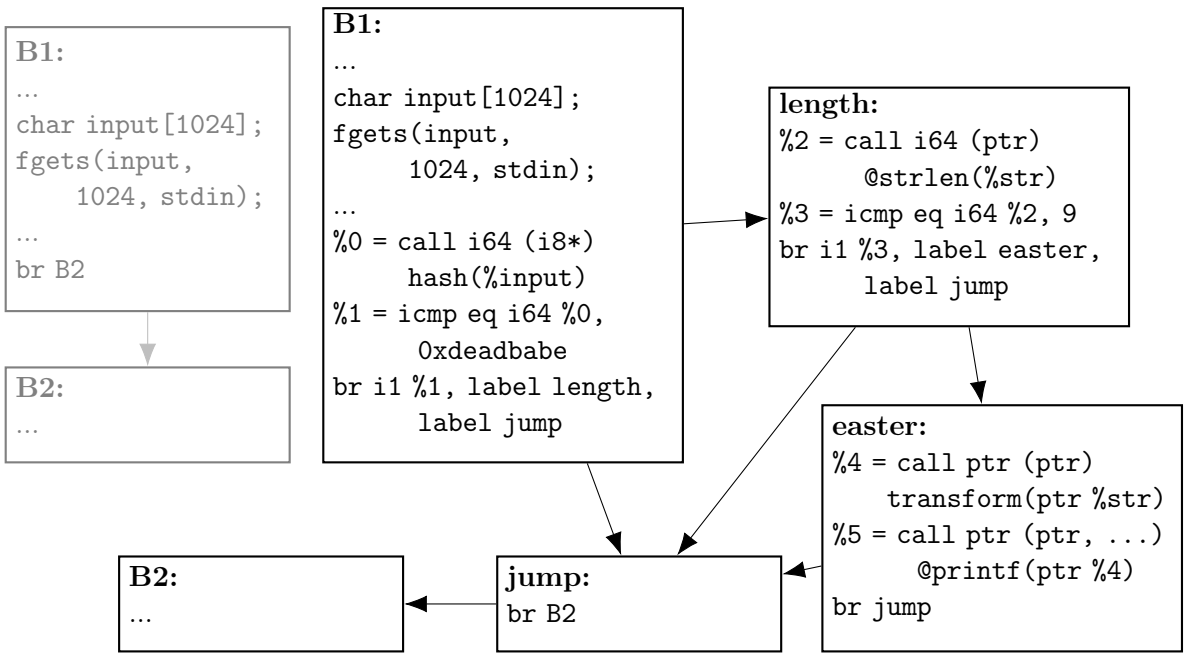


Figure 4.22: CFG Transformation for the SemaCall Watermark for `gets` and `fgets` with a check for string length

`time`

"`time`() returns the time as the number of seconds since the Epoch, 1970-01-01 00:00:00 +0000 (UTC)" [13]. It incorporates dynamic information in the form of the current time, making it possible to encode the watermark key as a timestamp. `gettimeofday` works similarly, except that the time is saved in a struct to which a pointer has to be passed as the first parameter [11]. We will dereference this pointer and access the entry for the seconds since the epoch, which allows us to use the same implementation for both functions, once the variable in which the time is saved has been accessed.

While they are the only functions in the selection, that do not deal with strings or text-based user input, the general structure can be kept and similar techniques applied. The hash value can be generated from the integer value `time` returns, like it was done for `atoi` etc. It can be compared with the application of the same hashing procedure on the timestamp-key and if they match, the easter-egg is triggered as before.

This is interesting, because it is no longer necessary, for the user to insert a certain prompt in a certain place, as was necessary with the other proposed watermarking techniques, but only a system global information has to match the key, making it possible to trigger all included watermarks at once by setting a single dynamic value. To avoid erroneous triggers of the watermark the timestamp-key should be in the past, making it impossible to trigger the watermark on up-to-date systems.

While there are functions in libc that allow retrieval of information about the micro-, nano-, or milliseconds of the current time, like `gettimeofday` itself, using only seconds is for the use-case more than enough since the time frame in which the watermark should be trigger-able should not be smaller than a couple of seconds. This could be achieved with `gettimeofday` as well, by just retrieving the seconds, which also has been implemented, however, the implementation after retrieving the seconds from the struct follows the same pattern as for `time`.

While there are tools on Linux or Unix-like systems, that allow to manipulate time, e.g. slowing down or stopping it (see `faketime` [14]), removing the necessity to make the time frame large enough for the watermark to be realistically trigger-able, in practice it is more convenient to be able to trigger the watermark without such tools. Therefore the timestamp returned by the function can be divided by the seconds the timeframe should contain (by integer division), as well as the timestamp-key. E.g. if the timeframe should be 1 minute long, the timestamp from `time` and the timestamp-key are both divided by 60s, making them matchable again after the hashing procedure.

However, because the key and the variables that trigger the watermark are no longer arbitrary long text values, but are limited to integer values, a new procedure to generate the watermark value is needed. It would be possible to use an integer-to-integer transformation as was done for the Register Aliasing Watermark, however, the limited throughput would decrease the usability of this implementation. Therefore a per-character genera-

tion using the time value as a seed would allow arbitrary throughput sizes, while still hiding the watermark key and value. This can be achieved by generating random instructions for each character again that all use the time value as their input.

While using the polynomial generation procedure in section 4.2.1 would be possible too, by using $f_i(x) = f(i + t)$, where t is the timestamp, i.e. eliminating the positional encoding of the character by just using the index position with the time as an offset and the watermark value characters as the target on the y-axis, this experiences the same disadvantages described in section 4.2.1. Therefore the char-by-char transformation has to be adapted to work with just the timestamp as input.

Since the hash generations are adapted to exactly generate the target value from the input or seed, this still works and only generates the correct watermark value for the correct timestamp. For each resulting character, random instructions are generated with at least one modulo operation s.t. if the timestamp-key divided by 60 is inserted, the watermark-value will be produced. By choosing random instructions and random values, the fact that the watermark can generate arbitrary strings will become less of a problem for every new insertion point, which makes it more important, that the watermark is inserted in many places. However, since much information is generated from little (an arbitrary large string from an integer), the watermark is already quite credible but also suffers the problem of easier detection by brute-forcing: once a set of viable timestamps that match the hash has been found, one could input different timestamps and observe the produced value. With a comparison to e.g. a dictionary, it would be possible to automatically reduce the set of produced values to semantic meaningful ones. This makes it easy to "guess" the correct watermark key-value pair.

```
#include <stdio.h>
#include <time.h>
int main(int argc, char** argv) {
        time_t t = time(NULL);
        printf("%ld\n", t);
}
```

```
$ ./a.out
1707843295
$ ./a.out
1707843300
watermark
$ ./a.out
1707843420
```

Figure 4.23: Example C program, compiled to include the time-watermark, triggered only in the the correct time frame.

The effect of the watermark is shown in figure 4.23. The watermark key specified during the compilation of the example program was the timestamp 1707843300 and the watermark triggers for every timestamp that equals the key in a timeframe of one minute (i.e. those timestamps divided by 60 match the key divided by 60). The modified control-flow-graph is shown in 4.24. No string length check is needed anymore and the hash generation works with the timestamp.

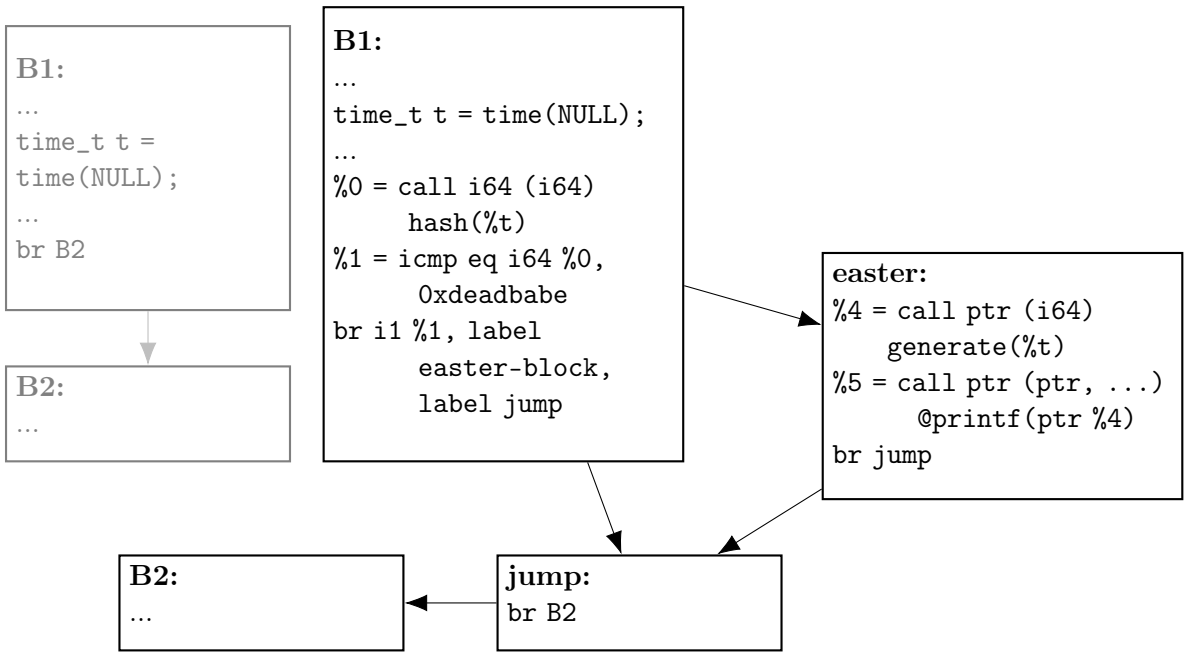


Figure 4.24: CFG Transformation for the SemaCall Watermark for `time`

4.2.2 Detection & Removal

Automatic Detection of the watermark in bytecode - as was possible for the *Register Expansion Watermark* 4.1.2 - is very difficult and in general impossible, if the insertion is combined with obfuscation techniques. While especially the code in the *easter-block* follows a common detectable pattern, the fact, that the watermark is inserted and interwoven in and with existing code and produces side effects, which are not generally differentiable from other print statements, help obfuscation to distribute the watermark more sparsely inside of the function and break the common pattern.

However, print statements that print strings constructed char-by-char with arithmetic transformations from user input are - depending on the functionality of the rest of the program code - possibly very rare and salient compared to the rest of the assembly. This helps manual-search for the watermark generation code. For the `time` and `atoi` implementation of the watermark, the hash code is only constructed from one integer value. If the watermark is detected once manually it is possible to reconstruct a set of integers that, with respect to integer overflow, produce the expected hash code when passed through the detected hash generation procedure.

The program can then be executed with each found integer in the set and the integer, which triggers the most easter-eggs should be the correct watermark-key. If it is ambiguous, the watermark key and value can't be identified certainly, but for the time watermark, probably one should be the correct combination, because wrong timestamp-

keys likely produce nonsensical strings.

A similar procedure can be used for the `fgets` watermark, however, the set of strings that produce a certain hash value is very large. Identifying the correct key-value combination is also unnecessary for the removal of the watermark, an input that produces the hash-code suffices.

If an input was found that triggers all watermark easter-eggs with a uniform value (which is not trivial, since wrong keys may produce different values at different insertion points, because of the modulo operations or integer overflow), the removal is straightforward: All print statements producing the identified value (if the value was correctly identified, this equals the watermark value, otherwise it is a random string) can be automatically marked and their basic-blocks and all branches to those basic blocks removed. Removing the string length check is also possible by removing the basic-block that only branches to the easter-egg block. This fully removes the watermark, without changing the normal behaviour of the program, but is only possible once at least one watermark insertion, and - more importantly - its hash generation and branch instruction could be identified, which is not an easy task once obfuscation has been applied to the assembly, as explained by Collberg et al [15]. Additionally, the randomly inserted hash transformations on the returned hash value additionally introduce an ambiguity that can prevent the identification of all easter-eggs at once.

In conclusion, the detection and removal depend on the uniqueness and information content of the watermark-key. As a consequence, the `atoi` and `time` implementations are easier to attack, since the hash generation makes it possible to narrow the candidates for the watermark-key down and because of the string-length check for the `atoi` watermark, only a fitting integer and random subsequent chars are necessary to trigger the watermark. The `time` watermark has in general few viable candidates for the watermark-key and less variation for the generated string in the easter-egg because the information is generated from only one 64-bit integer, which already can be narrowed down by the hash-generation, so try-and-error should produce better results then for especially the `fgets` implementation because in this case, the set of fitting keys has still enough variation per-character s.t. finding a semantic-relevant output, let alone the correct one, is complicated by the size of the viable candidates. A full-automatic detection or removal is - without knowing the watermark prior - however, not possible and requires a degree of human interaction to be partly automated.

If the watermark key and value are known prior, the detection is straight-forward since the watermark-value will just be printed on the standard output stream, once the key is presented in any possible file and stream reading scenario of the program (i.e. in all files that are opened, and in all streams like the standard input stream) for the `atoi` and `fgets` implementation and for the `time` implementation only the system time has to be set to the watermark-key. It is then even possible to fully automatically remove the watermark generation from the program as described prior.

Attacks on the watermark are also possible, however, generally not without breaking the functionality of the program. As a distortive attack, hash functions could be detected automatically and swapped out by different implementations. This would still keep the functionality of the hash function intact, while changing the returned value, causing the watermark key to possibly no longer trigger the easter-egg. However, this is only possible, if the original program did not expect the hash function to return certain values or if the generated value of the hash function is not used in an external data storage, e.g. the hash values could be used to index a table in a database.

Subtractive attacks that remove print statements that print strings that are generated by arithmetic calculations could be implemented as well. However, this would make command-line programs that e.g. deal with encryption unusable. In general, all programs that use the standard output to communicate with the user are at risk of being broken by attacks on the print function or the easter-egg block.

4.2.3 Evaluation

The test-suite is ineffective in measuring compilation-times and execution times since it uses only a few calls to the patched functions. In total all the patched functions have been called 17 times in all test programs, 15 times `atoi` and 2 times `gets`, `time` does not occur at all. 15 of these occurrences have been placed in the benchmarks.

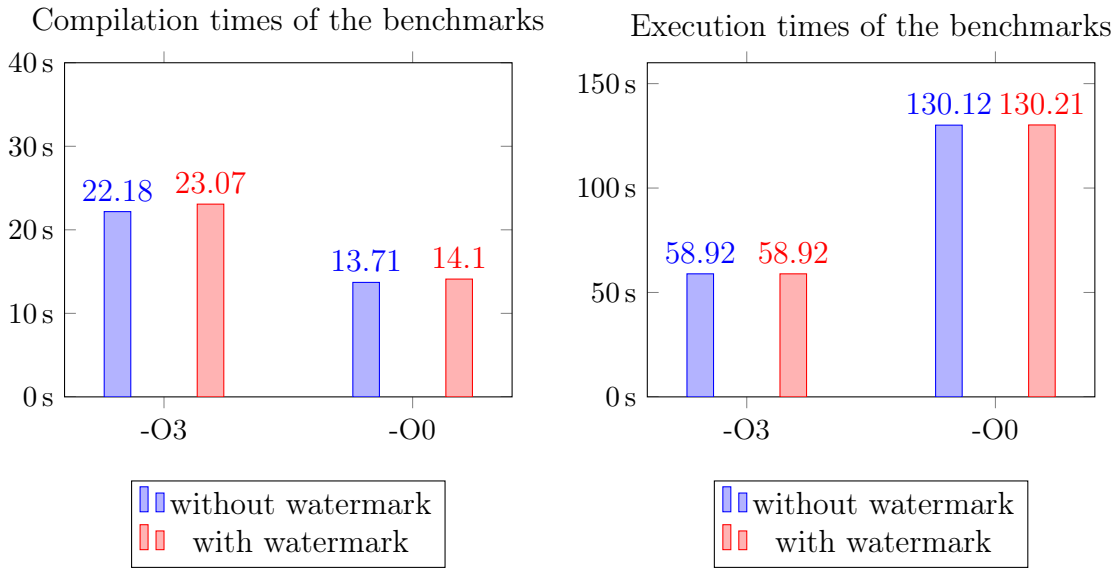


Figure 4.25: Compilation of the test-suite and execution of its benchmarks with and without the SemaCall Watermark

```c
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char** argv) {
  int n = atoi(argv[1])/2;
  for (int i = 0; i < n * n; i++)
      printf("%d", i - n/2);
  printf("\n");
}
```

```c
#include <stdio.h>
#include <time.h>
int main() {
  int n = time(NULL) / 240000;
  for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
      printf("%d", (i - n / 2) *
          (n / 2 - j));
}
```

```c
#include <stdio.h>
#include <stdlib.h>
int main() {
  char number[128];
  fgets(number, 128, stdin);
  int n = atoi(number) / 2;
  for (int i = 0; i < n*n; i++)
      printf("%d", i - n / 2);
}
```

Figure 4.26: Programs to benchmark the patching-libc-watermark. The call to libc is in no performance-critical block. Left to bottom: $\mathcal{P}_1^{atoi}$, $\mathcal{P}_1^{time}$, $\mathcal{P}_1^{gets}$

Since even a large performance overhead of the inclusion of the watermark during the compilation will be neglectable because of the small number of inclusions, we will only compile the benchmarks with the SemaCall watermark and additionally construct a small set of test-cases that heavily employ the patched library functions to evaluate the worst-case performance both during the compilation and runtime of the programs.

As the key-value pair for the watermark included during the compilation "12345test" is chosen as the watermark key and "watermark" as the value. Keep in mind, that the actual overhead heavily depends on the program, where the watermark is inserted and its usage of libc.

The difference between both the compilation- and runtime in the benchmarks in figure 4.25 is as expected very low because of the few insertion points, therefore the additional benchmarks tailored for the watermark could be more relevant.

The additional benchmarks that employ the libc-functions consist of 2 programs per patched function. The first program per function $\mathcal{P}_1^{function}$ uses the function outside of loops or recursion structures and is designed to provide a test-case where the runtime is not determined by the function call. If such a program receives a number on the command line or standard input, 15,000 is passed. To receive a good estimation each program is compiled and run 10 times both on a single thread, once with minimal optimizations and once with full optimizations enabled. For each program of the second

type $\mathcal{P}_2^{function}$ the libc-call is placed inside performance-critical loops to evaluate the performance difference in programs, where the runtime is governed by the libc-functions. $\mathcal{P}_2^{atoi}$ receives a set of 15,000 numbers between 0 and 99 and $\mathcal{P}_2^{gets}$ a stream on stdin where the first line contains the number of lines, which is 20,000,000, and each other line contains a random string. The set of programs compiled with the watermark are called $\mathcal{P}_{1,\mathcal{W}}$ and $\mathcal{P}_{2,\mathcal{W}}$, the ones without $\mathcal{P}_1$ and $\mathcal{P}_2$. It has to be noted, that these are artificial programs, that do not display a representability of general programs and are only used to estimate the worst-case scenario for the watermark.

The source code of the programs can be found in figure 4.26 and 4.27 in standard C. The compilation times of each program once unwatermarked and once watermarked, once with optimizations and once without can be found in figure 4.28, the execution times in figure 4.29. The -On prefix denotes the optimization level of LLVM.

```c
#include <stdlib.h>
int main(int argc, char **argv) {
  int n = atoi(argv[1]);
  int a = 0;
  for (int i = 0; i < n*n/2; i++)
      a += atoi(argv[
            (1 + i)%argc]);
  return a;
}
```

```c
#include <time.h>
int main() {
  int n = time(NULL) / 140000;
  int a = 0;
  for(int i = 0; i < n*n; i++)
      a += time(NULL) / 1000 *
        (i-n*n/2);
  return a;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
int main() {
  char number[128];
  fgets(number, 128, stdin);
  int n = atoi(number);
  int a = 0;
  for(int i = 0; i < n; i++){
    char str[128];
    fgets(str, 128, stdin);
    a += str[0];
  }
  return a;
}
```

Figure 4.27: Programs to benchmark the SemaCall watermark. The call to libc is in a performance-critical block. Left to bottom: $\mathcal{P}_2^{atoi}$, $\mathcal{P}_2^{time}$, $\mathcal{P}_2^{gets}$

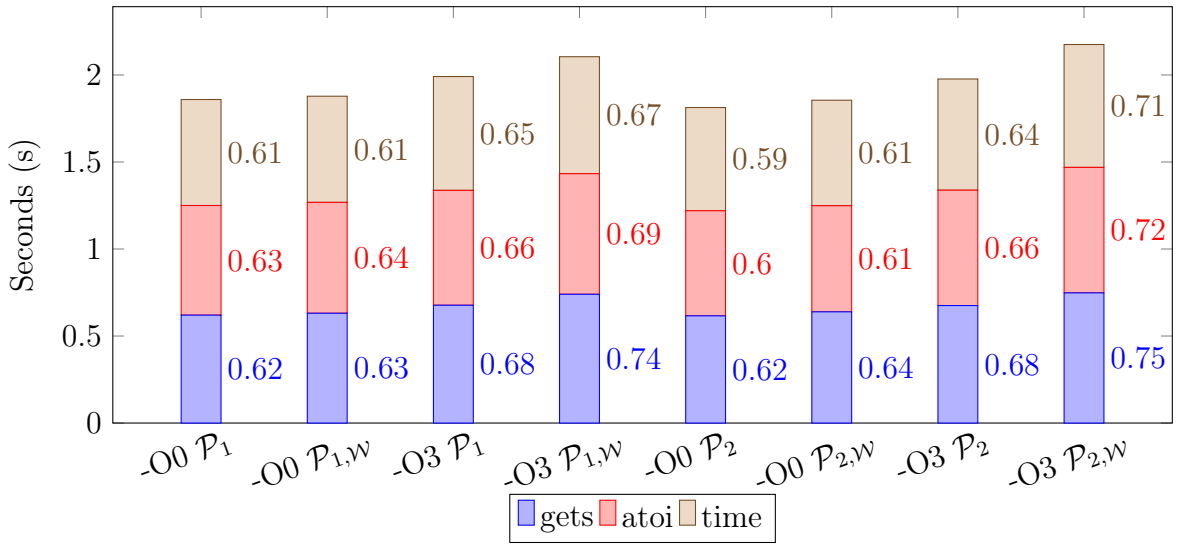


Figure 4.28: Per program compilation times of the SemaCall Watermark benchmarks

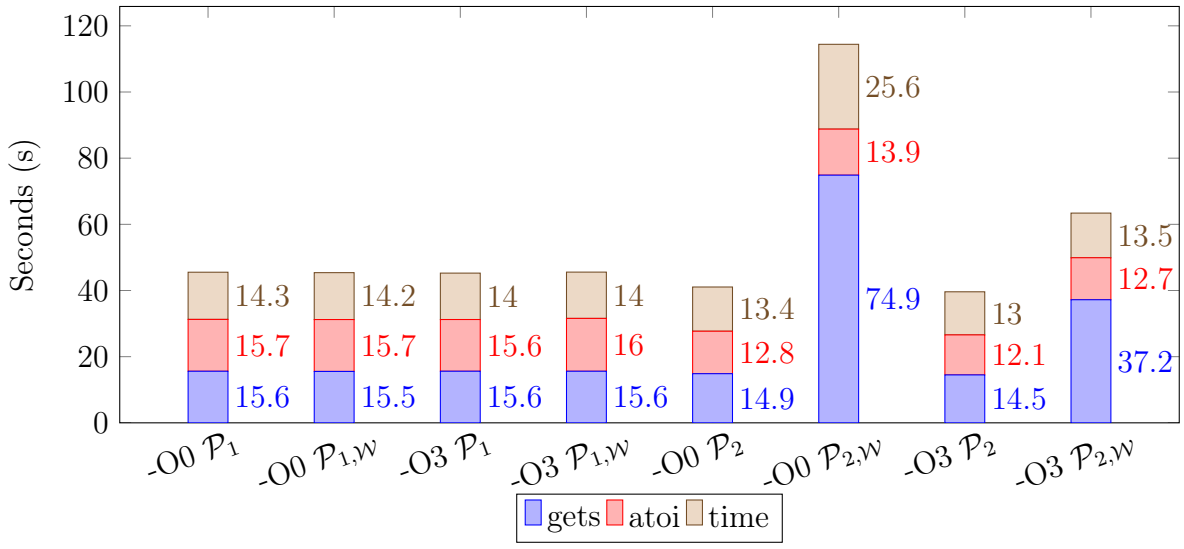


Figure 4.29: Per program execution times of the SemaCall Watermark benchmarks

While the compilation times show a measurable, but small overhead of the watermark inclusion, the execution times of the $\mathcal{P}_{2,\mathcal{W}}$ programs, are much higher. This is most visible in the unoptimized version, where the execution time with the watermark roughly doubled for $\mathcal{P}^{time}_{2,\mathcal{W}}$ and quintupled for $\mathcal{P}^{gets}_{2,\mathcal{W}}$. However, the execution overhead of $\mathcal{P}^{time}_{2,\mathcal{W}}$ in the optimized version is much lower and relatively small, so it should not impact most real programs.

This is not true for $\mathcal{P}^{gets}_{2,\mathcal{W}}$, here the execution times of the watermarked version is still

over 2.5 times that of the base program. This is most likely because of the overhead of the hashing function. But this benchmark includes only examples where the watermark has not been triggered, i.e. most input strings even have a different length then that of the watermark-key, nevertheless, the hash of them is calculated before the string length is checked. This will be the case for most applications of the watermark in practice, therefore it makes sense to optimize this scenario.

Currently, the string length is only checked once the hash value matches. This is due to similar implementations in other watermark implementations like `atoi`, where the hash generation is in $\mathcal{O}(1)$ for strings with length n and the string length check, which is in $\mathcal{O}(n)$, is more time-consuming, so for those functions it is sensible to omit it whenever possible.

But the hash generation of the `gets` watermark is also in $\mathcal{O}(n)$ and in practice even more time-consuming than the string length check because it uses more instructions per loop iteration. Therefore it would be sensible to switch the string length check and the hash value check, s.t. the short-circuit evaluation semantics only calculates the hash, if the string length matches. The modified CFG can be seen in figure 4.30 and the benchmark for the new version in figure 4.31. The execution times, especially for the unmodified version, are now much better. For the unoptimized version, it improved from 74.9s to 16.8s, increasing the performance by 445.8%.

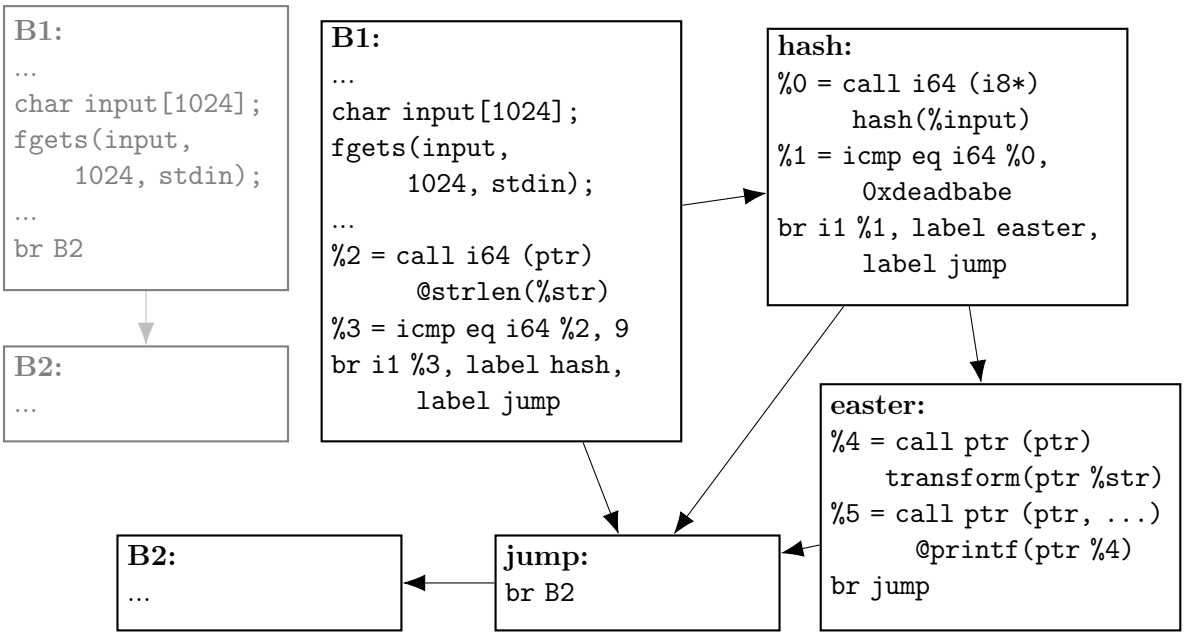


Figure 4.30: Optimized CFG Transformation for the SemaCall Watermark for `gets` and `fgets`

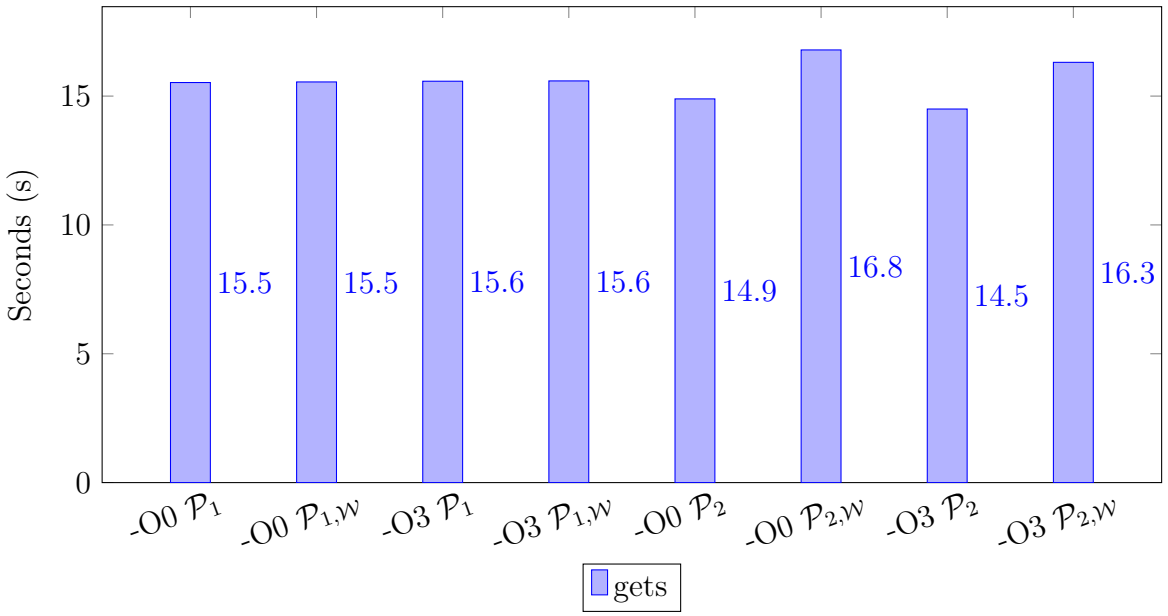


Figure 4.31: Per program execution times of the SemaCall Watermark benchmarks, with better `gets` implementation

The most relevant benchmark for programs where performance is an important factor is the difference between the -O3 $\mathcal{P}_2$ and -O3 $\mathcal{P}_{2,\mathcal{W}}$ programs (since such programs will always be optimized). The difference in execution time between the new implementation for $\mathcal{P}_{2,\mathcal{W}}^{gets}$ and $\mathcal{P}_2^{gets}$ is 12.4%, between $\mathcal{P}_{2,\mathcal{W}}^{atoi}$ and $\mathcal{P}_2^{atoi}$ 4.9% and between $\mathcal{P}_{2,\mathcal{W}}^{time}$ and $\mathcal{P}_2^{time}$ 3.8%. This shows, that the relative performance impact for especially `atoi` is not as high as expected since the hash generation was optimized by using only the integer parsed from the function itself, making the inclusion in arithmetic and logic-based functions, that don't rely on system resources, performance-wise feasible, at least if the hash generation can be simplified.

The implementation in `gets` shows that a string-based hash generation can have a large performance impact, but can be optimized if it is only carried out under certain conditions, e.g. if the string length matches. Keep in mind that those performance indicators are measured by programs that heavily utilize the given functions, especially the $\mathcal{P}_2$ programs, where the primary performance factor is the call to the patched functions. The performance impact will generally be much lower for most real-world applications, but as well will the number of possible inclusion places, making it less likely for the watermark to be included. For the implementation of `gets` the conditions to trigger (or not trigger) the easter-egg, i.e. the string length test and hash generation, are in $\mathcal{O}(n)$, however, n denotes the length of the input string, not the watermark throughput because both the hash function and the string length function scan the string to the end. The `time` and `atoi` implementation facilitate an easter-egg trigger that is in $\mathcal{O}(1)$ because they can use dynamic information specific to their semantic. If the easter-egg

is triggered, the generation of the watermark is in $\mathcal{O}(n)$ for all implementations where n stands for the watermark throughput as well as the input size.

However, these programs constitute no representing sample of real-world programs, which makes it difficult to predict the general impact of the watermark. To get an idea of how many times the watermark can be included in larger projects, LLVM [20] (just the base project, not clang or any additional components) as well as the GNU core utilities [6] have been compiled with the inclusion of the watermark.

	`atoi`	`gets`	`time`
Coreutils	31	0	42
LLVM	19	4	5

Figure 4.32: Number of inclusions of each libc watermark implementation in the GNU Coreutils

One could assume that the coreutils, as utilities that mainly perform system-interactions and command line tasks, would use such functions more often or at least one of `fgets` and `getline`, but according to the description of MaiZure's Projects "Decoded: GNU coreutils", the coreutils tend to use system-calls instead and additionally to the provided libc-functions: "coreutils often invoke syscalls through wrappers and helpers beyond those provided by libc. Many are linked through the Gnulib project." [22].

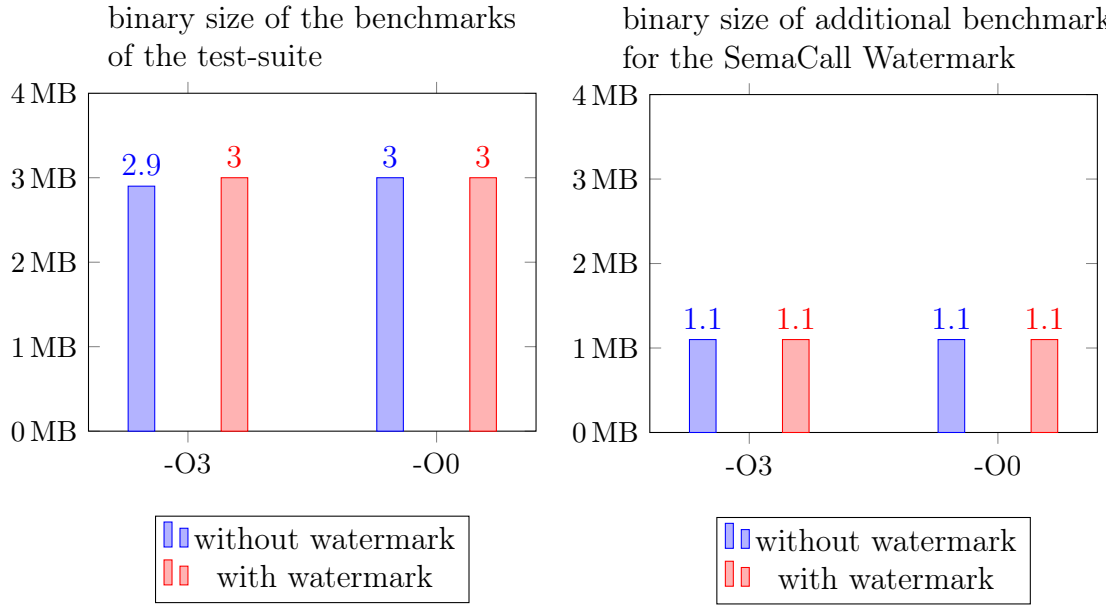


Figure 4.33: Size of the compiled benchmark binaries of the test-suite and the additional benchmarks with and without the SemaCall Watermark

It is also important to evaluate the binary size overhead of the programs where the watermark is inserted. To measure the "worst-case", the additional benchmarks are used, and the benchmarks of the test-suite are included additionally to see the difference in programs where the watermark is sparsely included. Each set of programs is compiled with and without the watermark, each one time with all optimizations enabled (-O3) and with all optimizations disabled (-O0). As a reference for a larger project, the binary size of the GNU coreutils was measured as well, but no difference was found. The results can be seen in 4.33.

The binary size overhead is very low, which is probably due to the low frequency of the patched functions. Even in the additional benchmarks the binary overhead is low since the actual patched function call - even if it is in a performance-critical point - substitutes only a small fraction of the program. Depending on the implementation, the binary size overhead grows with the size of the watermark. This is at least true for the *easter-block* in all implementations since it includes a per-character transformation. This could have been improved with the polynomial generation. However, since the polynomial also grows with higher throughput the advantage would be marginal.

Overall the watermark effectiveness can be evaluated with the discussed aspects on the six criteria:

- Credibility: Since arbitrary large watermarks are possible, this improves the credibility a lot. However, it is still possible to "fit" custom keys s.t. a different watermark would be generated by the inserted transformations. Since the original watermark will always fit through all insertion points, fitting it through one transformation is not enough and an attacker would have to fit it through all hash transformations, which becomes more and more difficult for each additional insertion point because of the uniqueness of the hash transformations. Therefore a watermark text that was inserted with this procedure is credible, especially if it was inserted at multiple points of the program.

- Robustness: Because of the side effects generated by the watermark and its connection to the rest of the program a full-automatic removal is not possible without damaging other parts of the program. However, we have shown, that a semi-automatic removal would be possible, if an attacker was able to reconstruct the correct watermark key and trigger all watermark values, however, this is not trivial since finding the correct key is not trivial and triggering all easter-eggs requires possibly the execution of all branches of the program (i.e. full coverage).

- Transparency: Complete automatic detection is simple if the key and value of the watermark are known and the dynamic environment of the program can be controlled. The watermark key has to be presented in all dynamic inputs (for `time` this is as easy as setting the system time, for the other functions everything from the standard input over inputs in graphical user interfaces to text files and network

connections become input possibilities for the key) and the value then just has to be detected on the standard output. Of course, this has to be adapted for each program since programs might deal with different kinds of input from different sources.

- Resilience: Some attacks can be applied to corrupt the watermark, however, non exist that - at least without additional knowledge of the implementation of the program - can corrupt the watermark without damaging the program. A distortive attack, that swaps out hash functions with other implementations was discussed, but this would break programs that rely on the predictability of hash functions for external data or constant values in the program. An additional subtractive attack, that attacks print statements would be possible, at least for all programs, that don't rely on the standard output for the usage and functionality of the program e.g. GUI applications. This could be counteracted by diversifying the side-effects to file-system interactions, graphical displays etc.

- Throughput: Each of the implementations was designed to enable arbitrary large watermark values, so the throughput is unlimited. However, this comes at the cost of increased performance and memory overhead.

- Runtime & Memory consumption: The runtime might be increased marginally in special worst-case scenarios, however, for many programs, the calls to libc are usually sparse and in performance uncritical blocks, s.t. the runtime overhead becomes barely noticeable. So the performance overhead is in general dependent on the watermark throughput and the program's usage of libc. The memory overhead is as large as the throughput since the key has to be transformed to the value, which is generated on stack memory.

For future work, the following aspects would be interesting to investigate:

- The per-character transformation could be improved, s.t. it folds over the key: For the generation of one result character, not a single character would be used, but all characters would be taken into account. This would make it increasingly difficult to construct specific new key-value pairs that generate a different watermark.

- Additional functions can be implemented, to increase the number of insertions and variety of the watermark. E.g. `strlen` could be implemented, which is used 1925 times in the GNU core utilities.

- The numeric stability of the polynomial generation could be improved, s.t. it can be used with a larger watermark throughput.

4.3 Sidedata Watermark

Collberg and Thomborsen's idea of representing watermarks by dynamic data structures [3] has been implemented by Palsberg et al. in "Experience with Software Watermarking" [18] by embedding a tree to represent an integer that encodes the watermark. However, this approach suffers from a very limited throughput size and is easy to remove by a simple dead code removal optimization. To protect the watermark against such attacks, Palsberg et.al. introduced conditions that use nodes of the tree and always evaluate to true or false. They then interweaved it inside the original code by moving existing statements to conditional branches that are always executed. This approach is very effective against dead code removal, but it is still vulnerable to constant propagation and constant folding, which could eventually remove the conditions, branches, and with them the dynamic structure that represents the watermark.

With our findings from the Register Expansion Watermark and the SemaCall Watermark we can improve upon this approach by reusing proven techniques, such as facilitating side effects in easter-eggs to prevent dead-code removal, using hashing techniques to detect the trigger for the watermark and generating the watermark during runtime to prevent the compromisation of the watermark in the binary code. Instead of using a elaborate tree structure to represent the watermark, we can use a simple stack-allocated array to store the watermark. Existing stack-allocated arrays are assumed to contain the watermark key, instructions are generated to test for its presence and if it has been detected, the watermark value is then generated in a new array. A side effect like e.g. a print statement has to be added to prevent dead code removal.

The *Sidedata Watermark* can be seen as an extension of the *SemaCall* Watermark for the `alloca` instruction and with that generally for stack-allocated strings. However, since it is not previously known if the array will actually contain a string, let alone if the data originates from user input, the semantics of the array is unknown, which differentiates it from the other implementations of the *SemaCall* Watermark.

4.3.1 Implementation

The watermark key is assumed to be in stack-allocated arrays, i.e. arrays allocated by the `alloca` instruction of LLVM, specifically those that have an 8-bit integer as their underlying type, to limit the inclusion to possible strings. First, the actual inclusion place of the watermark instructions has to be determined. Since adding it to the original block, would often only allow access to the uninitialized array since arrays might be filled by instructions and loops contained in the following basic blocks. Therefore it is more sensible to include it in the basic block of the last usage of the array. If LLVM already concluded the lifetime analysis, it is possible to use the calls to `llvm.lifetime.end` that mark the end of the lifetime of an array. In any case since that marker is just a call to a function that uses the array as a parameter, the last basic block in the topological order

of the dominator tree, which contains an instruction that references the array is taken to include the watermark in. Since this might result in the insertion of the watermark in a branch that is only unlikely executed, it could be more sensible to include the watermark in basic blocks that are still dominated by the definition of the array but dominate as many other basic blocks as possible, as a heuristic to maximize the probability of execution of the watermark. Furthermore, they could be limited to those blocks that are ordered after the block of last-usage in the topological sort to ensure that the array is filled. This changes the lifetime of the original array since it will be referenced in the included watermarking procedure. Our approach does not implement this technique, but it may be worth investigating the performance and detectability in future work.

The array is hashed at the end of the basic block of the last usage. The hash is then compared to a constant value that is stored in the binary code. Two additional basic blocks are generated: the *easter-block* and the *jump-block*. The *easter-block* contains the watermark generation and the *jump-block* the original last instruction of the basic block of the last usage of the array. At the end of this block, a conditional branch is inserted that jumps to the *easter-block* if the hash is equal to the constant value and otherwise to the *jump-block*. This equals the procedure of the *SemaCall* Watermark implementation of `gets` form section 4.2.1.

To generate the watermark from the key, the polynomial generation from section 4.2.1 is used. It fits better for the *Sidedata* Watermark than it did for the *SemaCall* Watermark, since higher throughput limits the detectability of the watermark: while it is still included for all possible 1-byte stack arrays, some of them might be too small to store the watermark. Therefore the limitation of 17 characters of the numerical stability of the polynomial is less of a problem than before.

The polynomial is generated by solving the linear program for the coefficient vector, the calculation of the polynomial is then compiled to LLVM IR as a callable function. Since this function does not change, as long as the watermark key and watermark value don't change, it can be reused in the same module, if it has been included already. This can reduce the binary size of the watermark inclusion, as long as the compiler does not decide to inline the function.

The *easter-block* then calls this function, by iterating over each character of the key, calculating the positional encoding of the key for its index, and passing it as the argument. The result is then stored in another stack-allocated array. This array can be allocated with the length of the key and value (including the terminating 0), however, this may not always be desirable, as it compromises an attribute of the key. Instead, the array can be allocated with the same length as the original allocation instruction plus 1 (for the terminating 0), in which the watermark key is expected to be. This additionally interweaves the watermark more with the original code and can be seen as a form of obfuscation. Afterward, in the last position of the array the terminating 0 is stored and the generated string can be printed on the standard output, issuing a side effect that

prevents dead code removal and allows easy extraction of the watermark. The terminating 0 is always added since for the case where the watermark is triggered erroneously or for arrays that are too small to contain the watermark message, the missing 0 could lead to problems with the print statement.

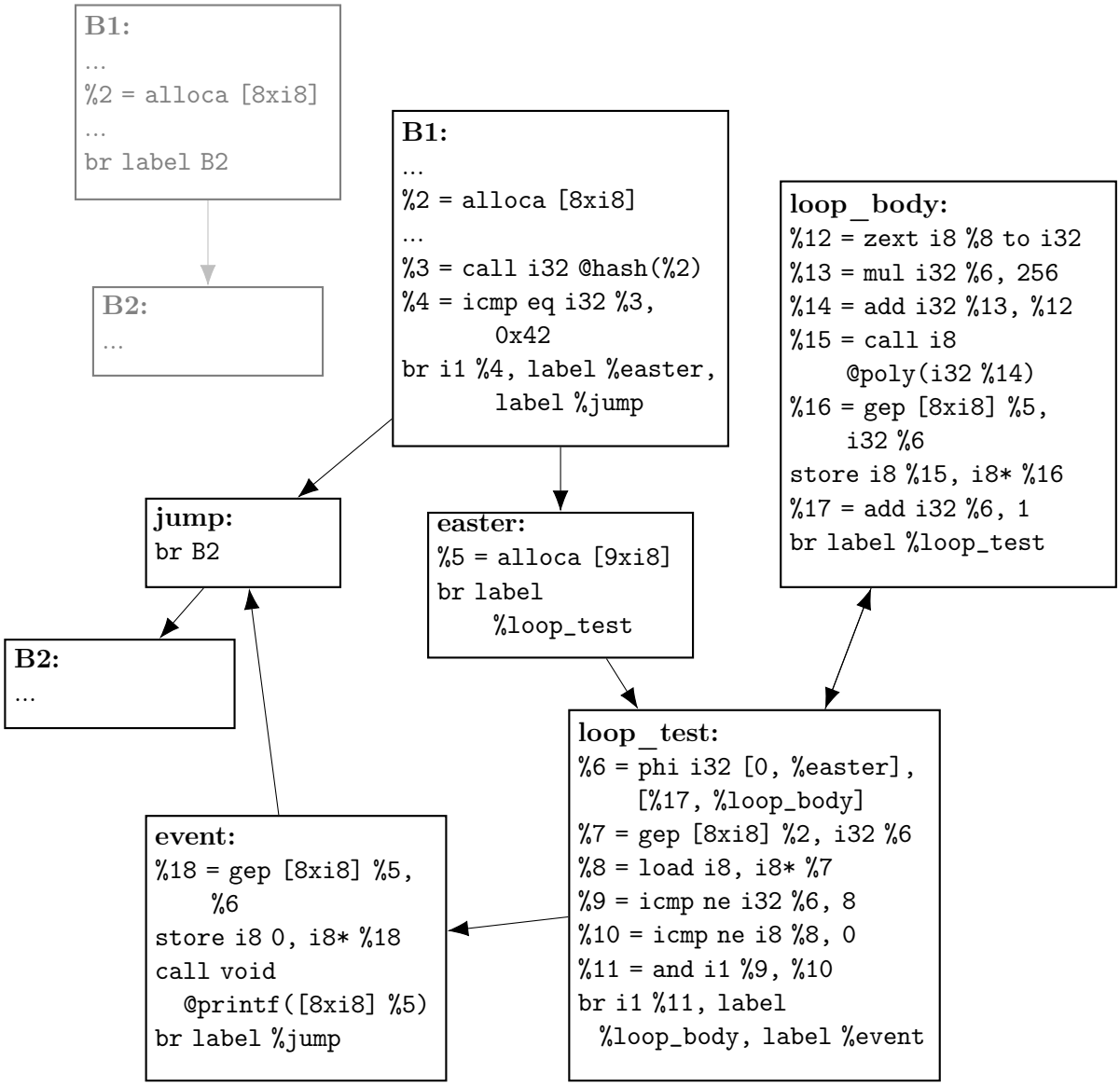


Figure 4.34: CFG Transformation of the Sidedata Watermark

An example inclusion of the Sidedata Watermark and the corresponding change in the control flow graph, can be seen in the figure 4.34. The original basic block $B1$, which contains the relevant allocation of a stack-allocated array with 8 1-byte integers (in the example we assume for simplicity, that the array is only used in $B1$), is extended by a

call to the hash function, a comparison of the hash to the constant that would have been produced if the key was inserted in the hashing function, and a conditional branch to the *easter-block* or the *jump-block*, depending on whether the hash matches the expected value.

In the original implementation, random instructions would have been applied to the result of the hash function %3, to make each comparison with the expected value as unique as possible, but for brevity this is omitted in the figure and expected to be implemented in the hash function itself. The *easter-block* contains the allocation of the array in which the generated watermark value is to be stored. This generation takes place in the loop blocks *loop_test* (head of the loop) and *loop_body* (body of the loop).

The *easter-block* branches to the *loop_test-block*, which loads the running index in %6, the current character from the original array in %8 and checks if the index is different than the array size and if the character is different from the terminating 0. If both conditions are met, the loop branches to the *loop_body-block*, which calculates the positional encoding of the character in %14 (by multiplying the index in %6 with 256 and adding the character value), calls the polynomial with it and stores the result in the generated array and increments the index. It then branches back to the *loop_test-block*

If the loop terminates, the *event-block* is reached, which stores the terminating 0 in the generated array, prints the array to the standard output, and branches to the *jump-block*. The *jump-block* is the last instruction of the original basic block and is used to continue normal control flow.

```
#include <stdio.h>
int main() {
        char input[1024];
        fgets(input, 1024, stdin);
        printf("%s\n", input);
}
```

```
$ ./a.out 12345word
12345word
$ ./a.out 12345test
12345test
watermark
```

Figure 4.35: Example C program, compiled to include the *Sidedata* watermark with the key "12345test" and value "watermark" and example inputs triggering the easter-egg

Figure 4.35 shows a simple program that includes the *Sidedata* watermark. Actually, this is the program from the `gets` implementation of the *SemaCall* watermark, in which the *Sidedata* watermark can be included as well, since it uses a stack-allocated array. The actual inclusion has nothing to do with the call to the `fgets` function, but includes the watermark after the print statement (the last usage of the array). The call to `fgets` is only used to fill the array.

4.3.2 Evaluation

Since the implementation is based on the *SemaCall* Watermark, the Robustness and Resilience inherit the same advantages and disadvantages (see section 4.2.2 and 4.2.3): A complete automatic removal is not possible, because the easter-egg causes side-effects that are not distinguishable from normal code and the trigger of the watermark is interwoven with the original code. The watermark can be removed by a manual analysis of the code, which can be automated to some extent, but is still a time-consuming process. The throughput is arbitrary as well, however, with increasing size, the probability of inclusion decreases, as the number of large enough array candidates, that can store the key, decreases. Since the watermark generation from key to value is done with a polynomial, the throughput is limited by the numerical instabilities, in our implementation up to 17 characters were possible.

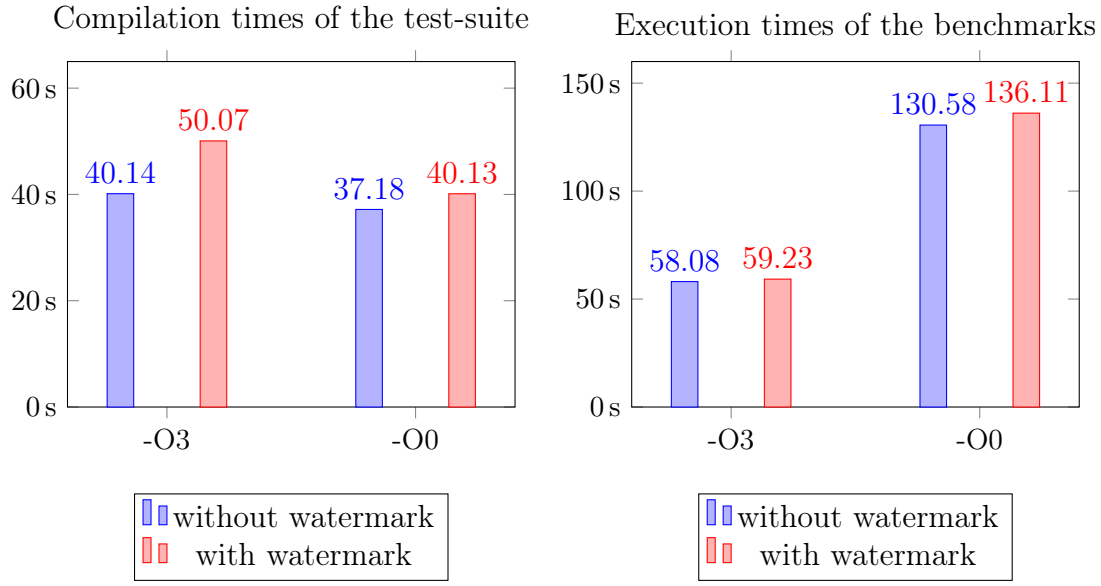


Figure 4.36: compilation of the test-suite and execution of its benchmarks with and without the sidedata watermark

The watermark was included in 23 functions in our test-suite, of which 13 inclusions are placed in the benchmarks. When compiling the GNU core utilities it was included in 157 functions, during the compilation of LLVM it was included 3217 times. The results of the compilation of the test-suite and the execution of the benchmarks can be seen in the figure 4.36. The compile times of the test-suite show a moderate increase, which is sensible, because the calculations for the polynomial generation and its compilation to LLVM IR, the insertion of new loops that have to be optimized again afterward and the modification of the existing data- and control-flow takes a non-neglectable increase

in the time necessary to compile especially small programs. The execution times of the benchmarks show nearly no increase when the watermark is included, which is partly because there were no false-positives i.e. no erroneous trigger of the watermark and because the watermark was included in only a few places. It also means, that the watermark was at most rarely included in performance critical structures (which would increase the performance overhead drastically if no string length check is present, as was shown in the evaluation of the `gets` implementation of the SemaCall Watermark, see section 4.2.3), which will not always be the case.

We adapted and compiled the Matrix Multiplication program from the evaluation of the *Register Expansion Watermark* (see section 4.1.3) with the *Sidedata Watermark*. To accomplish that, the result type was changed from integer to char (s.t. the watermark can be included in the 8-bit result array). This nearly showed no runtime difference since the LLVM lifetime analysis marked the end of the lifetime of the result array at the end of the loops, which included the watermark there as well, outside of any performance-critical blocks.

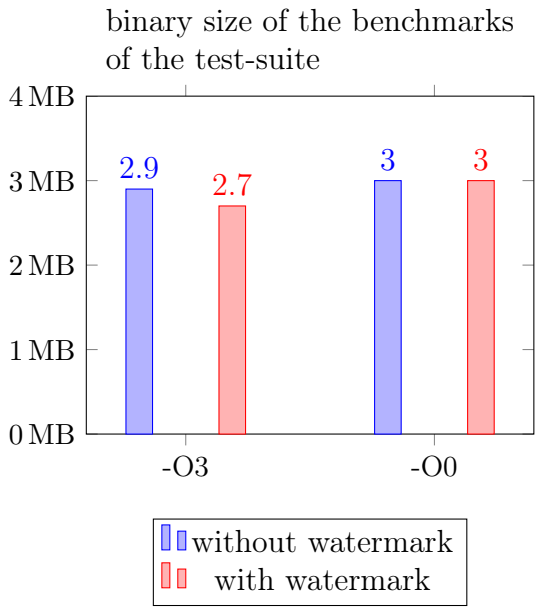


Figure 4.37: Size of the compiled benchmark binaries of the test-suite with and without the Sidedata Watermark

The binary sizes were again measured on the test-suite and the core utilities and listed in figure 4.36. The reduced binary size for the high optimization level shows that the watermark either prevents optimizations like vectorization, inlining, or loop unrolling that could increase the binary size and/or helps optimizations that reduce the binary size. In conclusion, we can evaluate the watermark with our criteria:

- Credibility: In contrast to the implementation of the *SemaCall* watermark, the watermark is limited by the numerical stability of the polynomial, which was in our experiments limited to 17 characters. However, with additional numerical optimizations, this could be improved. Overall the throughput will stay limited to a certain size, however, other limitless transformation procedures like the per-character transformation from e.g. the `gets` implementation of *SemaCall* could be used to allow arbitrary throughput as well. The relatively large throughput again reinforces credibility. Since the semantics of the procedure are the same as for the *SemaCall* watermark, it also suffers from the problem, that a custom key can be fitted to produce a certain value, which also becomes more difficult for each additional insertion. Therefore a watermark text that was inserted with this procedure is credible, especially if it was inserted at multiple points of the program.

- Robustness: The robustness is the same as for the *SemaCall* watermark. The side-effects prevent a full automatic removal of the watermark. The pattern is arguably less salient because it does not only appear with the presence of functions from a specific library but every time a suitable stack-array is allocated. It also does not directly follow the allocation but is placed at the end of the original lifetime of the array, which further obfuscates it. The same semi-automatic removal procedure as explained for the *SemaCall* watermark in section 4.2.2 is possible.

- Transparency: To trigger the watermark, the key has to be present in the relevant stack-allocated arrays. In any case, this can be taken care of with a debugger, if the insertion places have been known prior. Otherwise, if such arrays have been filled with user-input or dynamic input from the system, these input sources can be manipulated to present the key. Since the watermark does not directly deal with the origin of the data, it is more complicated and requires more knowledge about the program to trigger the watermark. Once it is triggered, the detection is again as easy as detecting a string on the standard output.

- Resilience: The distortive and subtractive attacks from the *SemaCall* watermark and similar ones are again possible. All needed a good knowledge of the program and were only possible in certain cases to ensure that the functionality is not corrupted by the attack.

- Throughput: As mentioned for the credibility, the throughput is limited by the numerical stability of the polynomial, which was in our case 17 characters. Larger throughput could hurt the places in which the watermark can be reliably triggered, since arrays that have too little space to store the watermark key, can't trigger it.

- Runtime & Memory consumption: The runtime overhead was low in all of our test-cases. However, it is possible, that the watermark can be included in performance-critical places where the hashing of a large array can impact the runtime drastically,

as it was reported for the `gets` implementation of the *SemaCall* watermark. Due to the insertion of the watermark at the end of the lifetime determined by LLVM, this is most of the time unlikely. Memory consumption might double on the stack in the worst-case scenario, which might be a problem for recursive functions, but should be generally feasible since large arrays are usually allocated in heap memory (because of the limited size of the stack and the problems that may arise for deep function-calling-chains like for recursive functions).

Further Work could include developing more complex data-structures in which the watermark could be encoded, as was used and shown by Palsberg et al. in "Experience with Software Watermarking" [18]. These could additionally benefit the Robustness and Resilience of the watermark. Additionally it would be worth investigating if there are better candidates for the basic block in which the watermark will be inserted.

5 Conclusion & Future Work

We've developed three novel dynamic watermarking techniques. Each one used in some way hashing techniques to transform a key present in the dynamic state of the program to a value that represents the watermark. This means, that instead of inserting a watermark directly in the program, the watermark is encoded by a transformation procedure that is able to produce the value for a certain prompt.

The first one, the *Register Expansion Watermark* expanded Values from 32-bit to 64-bit and included additional instructions to generate and inherit the watermark value from value to value by their calculations. The key has to be inserted and the value retrieved from the register and stack contents during runtime with a debugger. While it does not modify the semantics of the program, this comes with the caveat of a low robustness, since an automatic removal can be implemented, that detects unused upper 32-bits of 64-bit values and lowers them back to 32-bit. The throughput which allows the encoding of only 1-2 bytes (depending on if the key counts into the throughput) could be improved by using different parts of a larger watermark at different inclusion places in the program, however, the throughput is then limited by the possible inclusion places of the program, which means, that some programs won't be able to encode certain throughput sizes. Overall the *Register Expansion Watermark* is a good choice for programs in which the watermark has to be encoded as low-key as possible and if the throughput is negligible. It introduces no side effects and uses only basic instructions without the neccessity of libraries.

The *SemaCall Watermark* used calls to functions for which an implementation of the watermark has been defined. The dynamic information and possibly the semantics of the parameters can be used to detect if a key is present by applying a hash function and checking for the correct value. Once the key is detected, hash transformations are applied to transform the key to the watermark value and produce a side effect that signals the presence of the watermark, in our implementation a simple print statement. Normal execution of the program is then continued. The detection with hash functions was used to prevent literal encoding of the watermark and make each trigger unique s.t. reconstruction of the watermark becomes more difficult with each additional insertion point. However, this enables false-positives for collisions in the hash function, which means unintentional triggering of the easter-egg and watermark, which changes the semantics of the program and makes it unusable in some cases. The watermark throughput depends on the types of functions. While mostly arbitrarily large strings are desirable, we've implemented a time-based approach, that encodes the watermark

for a specific timestamp of the system time, which except for the throughput showed many advantages, e.g. it is very simple to trigger and stable. The performance of the watermark depends on the usage of the selected functions, for our selection of the libc library the runtime overhead was even in the worst case feasible and in our test-suite barely noticeable. How many times and in how many branches the watermark can be embedded in the program is dependent on the program's usage of the selected functions. Overall the *SemaCall Watermark* is a good choice if a high throughput with low average overhead is important and if it is known that the program uses the functions for which the watermark was implemented enough to credibly include it. The watermark expects that the standard library is linked against the program to use procedures like `strlen`. If a program depends on exact behavior on the standard output, false-positives could destroy its reliability and the watermark might be not a good choice for such cases.

The *Sidedata Watermark* is an adaption of the *SemaCall* watermark for arbitrary 1-byte integer arrays and it targets strings. It uses the hash function from the *SemaCall* watermark to detect the presence of the watermark which is in addition limited by the size of the allocated array (because non-string arrays are not terminated with 0). For the generation, a polynomial is compiled which allows to transform each character by mapping each character of the key on the x-axis and the corresponding result of the polynomial as the resulting character of the value on the y-axis. While it's inclusion depends on the usage of stack-allocated arrays in the program, this is a more generic assumption than the usage of specific library functions. In theory, it could have a moderate performance impact, however, all our test-cases showed that it is most often inserted in places, where the performance impact is minimal.

Each of the watermarks enables inclusion in various places and encodes the watermark in different forms. Each of the procedures is a proof-of-concept and expandable. For the *Register Expansion Watermark*, the inheritance of the watermark has to be implemented for each instruction type, since effects on the upper bytes have to be counteracted. We've implemented it for various instructions, but many more can be used to increase the includability of the watermark. Future work also includes a study about the effectiveness of splitting a larger watermark into many 1-byte key-value pairs and using each pair for a different inclusion place to increase the throughput of the watermark.

The *SemaCall Watermark* depends on the implementations for often-used functions. We only scratched the surface of libc with the implementation of the 8 functions, a study on other largely used functions like `strlen` and `strcmp` could be interesting but also functions that use different system information e.g. the version of the operation system or similar could be developed. The performance difference between the two transformation procedures (the per-character hashing transformation and the polynomial) could be investigated.

The *Sidedata Watermark* uses simple arrays to encode the watermarks as strings. More complex datastructures could be used to encode the watermarks and further disguise

them in the binary. Furthermore different heuristics and techniques to determine the basic-block in which the watermark should be embedded can be investigated.

The actual effect of obfuscation techniques and studies on the robustness and resilience of the watermarks in practice could be deducted to substantiate the theoretical considerations. The automatic removal for the *Register Expansion Watermark* could be implemented and its effectiveness tested in combination with obfuscation and temperproofing techniques.

Overall each of the watermarks can be combined with others and possibly they could cover the disadvantages of each other. Since they are written as LLVM passes that can be dynamically loaded into the compiler, they facilitate an extension of the Dynamic Watermarking Framework proposed by Novac et al. in "LLWM & IR-Mark: Integrating Software Watermarks into an LLVM-based Framework" [17] and allow combinations with various other watermarks. A synergy between watermarks is possible as well, e.g. the *Register Expansion Watermark* could be inserted into the polynomial of the *SemaCall* watermark. The effectiveness could be studied as well, possibly with other dynamic watermarks.

List of Algorithms

1 Marking of Candidates for Register Expansion 15
2 Expanding Candidates to 64-bit for Register Aliasing 16
3 Transformation of watermark key to value 17
4 Insertion of the Watermark 21
5 Generic form of hash functions 43

List of Figures

3.1 Compilation Pipeline with the Watermark as a Transformation Pass . . . 9

4.1 Illustration of 64-bit register padding and position of watermark 12
4.2 Example code and data flow graph for the transformation of the Register Padding Watermark 14
4.3 Color grading of the data flow graph from blue to red of the iteration in which each node was added as a candidate for the Register Expansion Watermark 15
4.4 Visualization of the data flow graph for the expansion of the candidates to 64bit for the Register Expansion Watermark 16
4.5 Visualization of the data flow graph after the insertion of the Register Expansion Watermark for the example from figure 4.2 19
4.6 Example LLVM IR before and after the watermark insertion 22
4.7 Example of user input being directly passed to a function 23
4.8 Example in `gdb`, the watermark key `0xbadc0ffe` was manually inserted in `rdi` 23
4.9 Example in `gdb`, the watermark value `0xdeadbeef` has been generated in `rcx` 23
4.10 Percentile of generated left-shift instructions, constants with an absolute value larger than 1024 and the constant value 32 in the affected programs in the LLVM test-suite 25
4.11 Compilation of the test-suite and execution of its benchmarks with and without the Register Expansion Watermark 27
4.12 Size of the compiled benchmark binaries of the test-suite with and without the Register Expansion Watermark 28
4.13 Execution time of Matrix Multiplication with and without the *Register Expansion Watermark* 29
4.14 pattern of a patchable function call in the SemaCall Watermark 33
4.15 CFG Transformation for the SemaCall Watermark 35
4.16 Plotted polynomial transforming the string "test12345" to "watermark" . 38
4.17 Time per string length for the construction of the key-value transformation polynom 39
4.18 CFG Transformation for the SemaCall Watermark for `atoi`, `atol` and `strtol` with a check for string length 42

4.19 Example C program, compiled to include the atoi-watermark and example inputs triggering the easter-egg . . . 42
4.20 Example C program, compiled to include the fgets-watermark with the key "12345test" and value "watermark" and example inputs triggering the easter-egg . . . 44
4.21 CFG Transformation for the SemaCall Watermark for `getline` with a check for string length . . . 45
4.22 CFG Transformation for the SemaCall Watermark for `gets` and `fgets` with a check for string length . . . 45
4.23 Example C program, compiled to include the time-watermark, triggered only in the the correct time frame. . . . 47
4.24 CFG Transformation for the SemaCall Watermark for `time` . . . 48
4.25 Compilation of the test-suite and execution of its benchmarks with and without the SemaCall Watermark . . . 50
4.26 Programs to benchmark the patching-libc-watermark. The call to libc is in no performance-critical block. Left to bottom: $\mathcal{P}_1^{atoi}$, $\mathcal{P}_1^{time}$, $\mathcal{P}_1^{gets}$ . . . 51
4.27 Programs to benchmark the SemaCall watermark. The call to libc is in a performance-critical block. Left to bottom: $\mathcal{P}_2^{atoi}$, $\mathcal{P}_2^{time}$, $\mathcal{P}_2^{gets}$ . . . 52
4.28 Per program compilation times of the SemaCall Watermark benchmarks 53
4.29 Per program execution times of the SemaCall Watermark benchmarks . . 53
4.30 Optimized CFG Transformation for the SemaCall Watermark for `gets` and `fgets` . . . 54
4.31 Per program execution times of the SemaCall Watermark benchmarks, with better `gets` implementation . . . 55
4.32 Number of inclusions of each libc watermark implementation in the GNU Coreutils . . . 56
4.33 Size of the compiled benchmark binaries of the test-suite and the additional benchmarks with and without the SemaCall Watermark . . . 56
4.34 CFG Transformation of the Sidedata Watermark . . . 61
4.35 Example C program, compiled to include the *Sidedata* watermark with the key "12345test" and value "watermark" and example inputs triggering the easter-egg . . . 62
4.36 compilation of the test-suite and execution of its benchmarks with and without the sidedata watermark . . . 63
4.37 Size of the compiled benchmark binaries of the test-suite with and without the Sidedata Watermark . . . 64

Bibliography

[1] The Software Alliance. "Seizing Opportunity Through License Compliance". In: *BSA GLOBAL SOFTWARE SURVEY* (2016).

[2] Md Asikuzzaman and Mark Pickering. "An Overview of Digital Video Watermarking". In: *IEEE Transactions on Circuits and Systems for Video Technology* PP (June 2017), pp. 1–1. DOI: 10.1109/TCSVT.2017.2712162.

[3] Christian Collberg and Clark Thomborson. "Software Watermarking: Models and Dynamic Embeddings". In: Auckland, New Zealand, May 2000.

[4] J.F. Epperson. *An Introduction to Numerical Methods and Analysis*. Wiley, 2014. ISBN: 9781118730966. URL: https://books.google.de/books?id=mOLAAgAAQBAJ.

[5] Anton Ertl Fabian Schmied M. *AMD64-Assembler-Handbuch*. https://www.complang.tuwien.ac.at/ubvl/amd64/amd64h.html. 2023.

[6] Inc. Free Software Foundation. *GNU core utilities*. https://www.gnu.org/software/coreutils/. accessed: 2023-11-19. 2020.

[7] Michael Kerrisk. *atoi(3) — Linux manual page*. https://man7.org/linux/man-pages/man3/atoi.3.html. accessed: 2024-02-03. 2023.

[8] Michael Kerrisk. *fgetc(3) — Linux manual page*. https://man7.org/linux/man-pages/man3/fgets.3.html. accessed: 2024-02-03. 2023.

[9] Michael Kerrisk. *getline(3) — Linux manual page*. https://man7.org/linux/man-pages/man3/getline.3.html. accessed: 2024-02-03. 2023.

[10] Michael Kerrisk. *gets(3) — Linux manual page*. https://man7.org/linux/man-pages/man3/gets.3.html. accessed: 2024-02-03. 2023.

[11] Michael Kerrisk. *gettimeofday(2) — Linux manual page*. https://man7.org/linux/man-pages/man2/gettimeofday.2.html. accessed: 2024-02-03. 2023.

[12] Michael Kerrisk. *strtol(3) — Linux manual page*. https://man7.org/linux/man-pages/man3/strtol.3.html. accessed: 2024-02-03. 2023.

[13] Michael Kerrisk. *time(2) — Linux manual page*. https://man7.org/linux/man-pages/man2/time.2.html. accessed: 2024-02-03. 2023.

[14] Ubuntu Manuals. *Ubuntu Manpage: faketime - manipulate the system time for a given command*. https://manpages.ubuntu.com/manpages/trusty/man1/faketime.1.html. accessed: 2024-02-07. 2019.

[15] J. Nagra and C. Collberg. *Surreptitious Software: Obfuscation, Watermarking, and Tamperproofing for Software Protection*. Addison-Wesley Software Security Series. Pearson Education, 2009. ISBN: 9780132702034. URL: https://books.google.de/books?id=mig-bH3u0Z0C.

[16] CBC News. *The pros, cons, and future of DRM*. https://www.cbc.ca/news/science/the-pros-cons-and-future-of-drm-1.785237. accessed: 2024-02-20. Aug. 2009.

[17] Daniela Novac, Christian Eichler, and Michael Philippsen. "LLWM & IR-Mark: Integrating Software Watermarks into an LLVM-based Framework". In: *Proceedings of the 2021 Research on Offensive and Defensive Techniques in the Context of Man At The End (MATE) Attacks*. Checkmate '21. Virtual Event, Republic of Korea: Association for Computing Machinery, 2021, 35–41. ISBN: 9781450385527. DOI: 10.1145/3465413.3488576. URL: https://doi.org/10.1145/3465413.3488576.

[18] Jens Palsberg, Sowmya Krishnaswamy, Minseok Kwon, Di Ma, Qiuyun Shao, and Yi Zhang. "EXPERIENCE WITH SOFTWARE WATERMARKING". In: *CERIAS Tech Report 2004-30* (2004).

[19] LLVM Project. *LLVM Language Reference Manual*. https://llvm.org/docs/LangRef.html. accessed: 2023-11-19. 2023.

[20] LLVM Project. *LLVM Project*. https://github.com/llvm/llvm-project. accessed: 2023-11-19. 2023.

[21] LLVM Project. *llvm-test-suite*. https://github.com/llvm/llvm-test-suite/. 2003.

[22] MaiZure's Projects. *Decoded: GNU coreutils*. https://www.maizure.org/projects/decoded-gnu-coreutils/. accessed: 2024-03-19. Sept. 2019.

[23] Medahalli Ramakrishna and Justin Zobel. "Performance in Practice of String Hashing Functions". In: *International Conference on Database Systems for Advanced Applications*. 1997. URL: https://api.semanticscholar.org/CorpusID:8250194.

[24] L.R. Turner, United States. National Aeronautics, and Space Administration. *Inverse of the Vandermonde Matrix with Applications*. NASA technical note. National Aeronautics and Space Administration, 1966. URL: https://books.google.de/books?id=e2BIZ5P7Dk0C.

[25] Shweta Wadhera, Deepa Kamra, Ankit Rajpal, Aruna Jain, and Vishal Jain. *A Comprehensive Review on Digital Image Watermarking*. 2022. arXiv: 2207.06909 [cs.MM].

[26] andrewchambers and Frunze Alexey. *c-testsuite*. https://github.com/c-testsuite/c-testsuite/. 2018.